



Go Crew

10 – 14 ноября

Нестандартный Go

Данил Губанов
Точка Банк

Доклад
Сетевые эмуляторы:
от физических к виртуальным

точка банк Selectel





Даня из Точки

Партнёрский кабинет

Backend-разработка
(Go/Python)

Песочница сетевых
окружений



Партнёрство с <УрФУ>

- 1 Курс разработки на Python
- 2 Стажировка
- 3 Проведение и участие в конференциях
- 4 Платформа песочницы сетевых окружений по курсам:
 - Построение масштабируемых корпоративных сетей
 - Компьютерные сети (традиционный курс)
 - Введение в программное управление компонентами коммуникационных инфраструктур (запускается)

- 1 Работаем с сетью и GO
- 2 Состав идеальной виртуалки
- 3 Образы железак
- 4 Платформы виртуальных железак

Прыгаем в сети



Распаковка устройства

точка банк

Распаковка устройства

точка банк



Распаковка устройства

точка банк



Распаковка устройства

точка банк



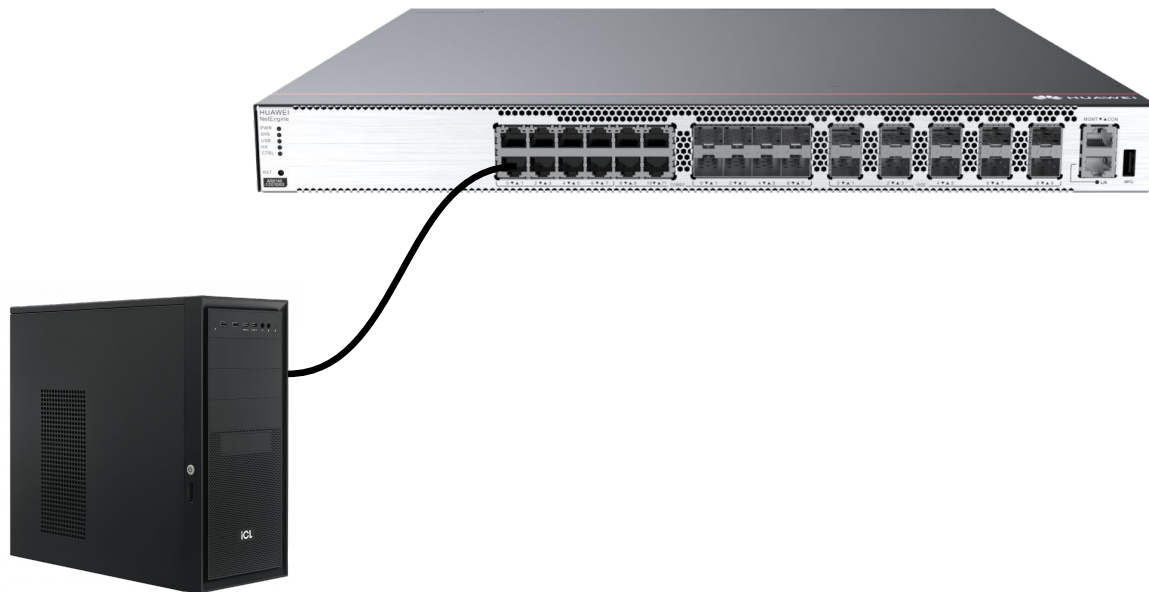
Подключение двух устройств

точка банк



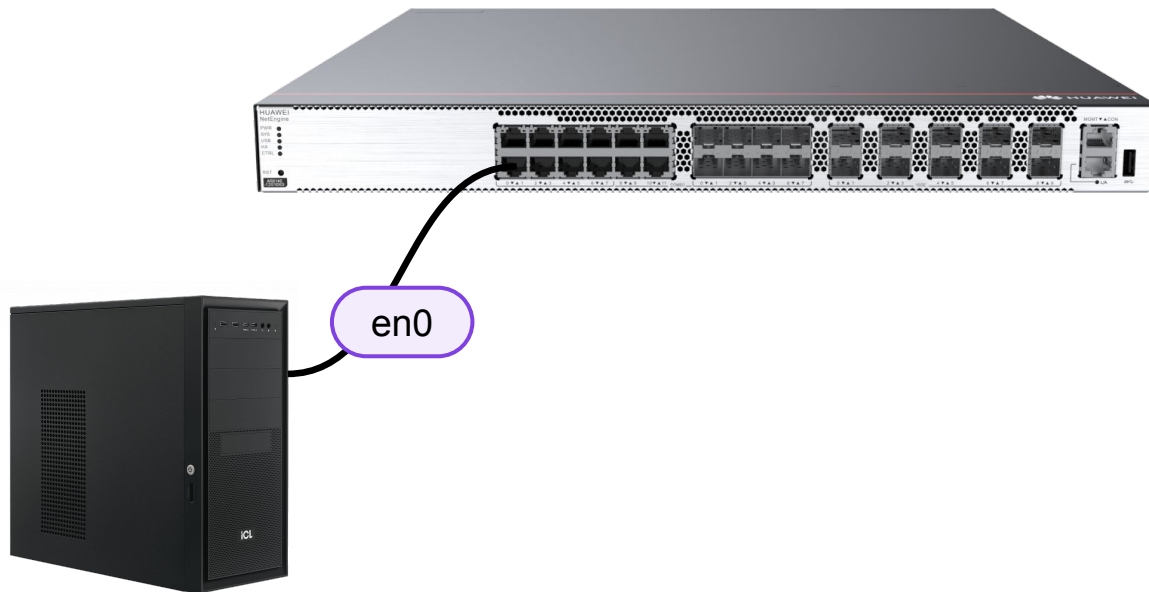
Подключение двух устройств

точка банк



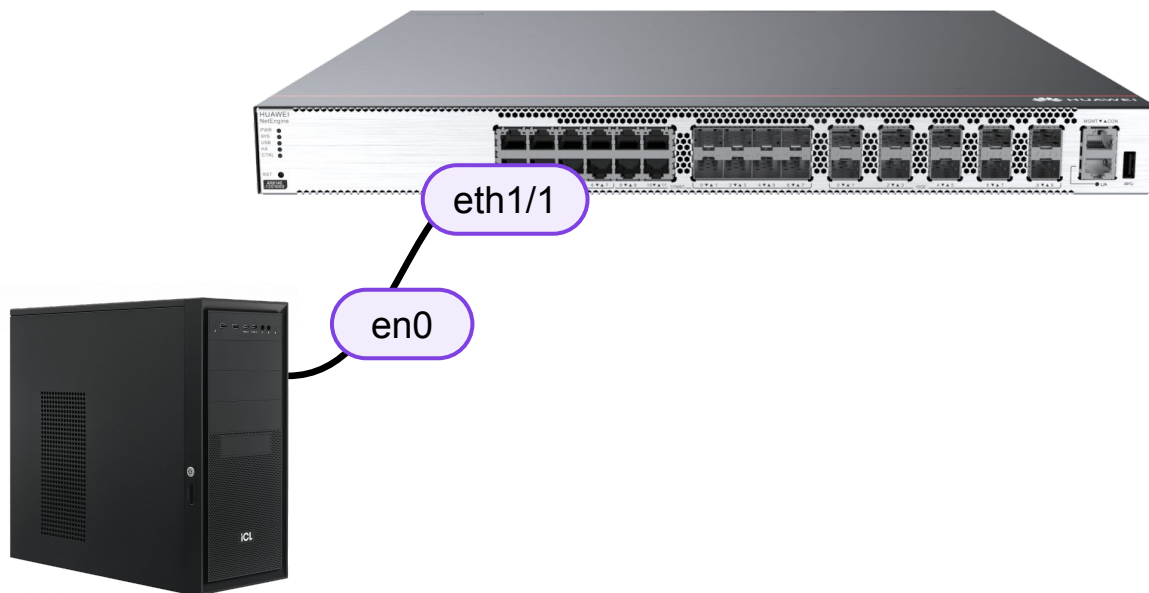
Подключение двух устройств

точка банк



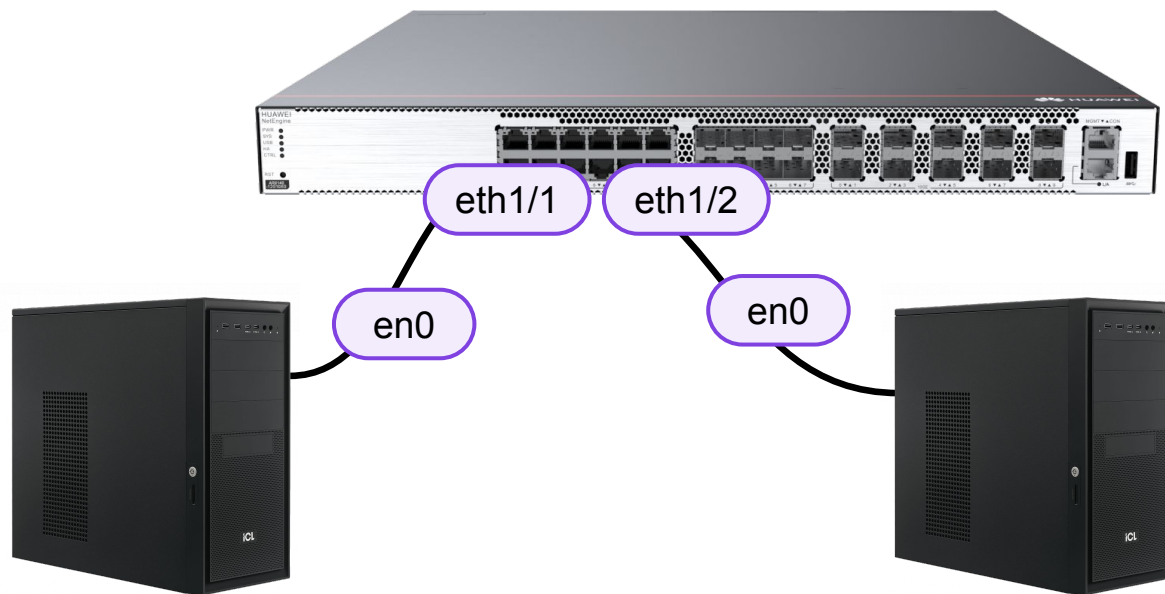
Подключение двух устройств

точка банк



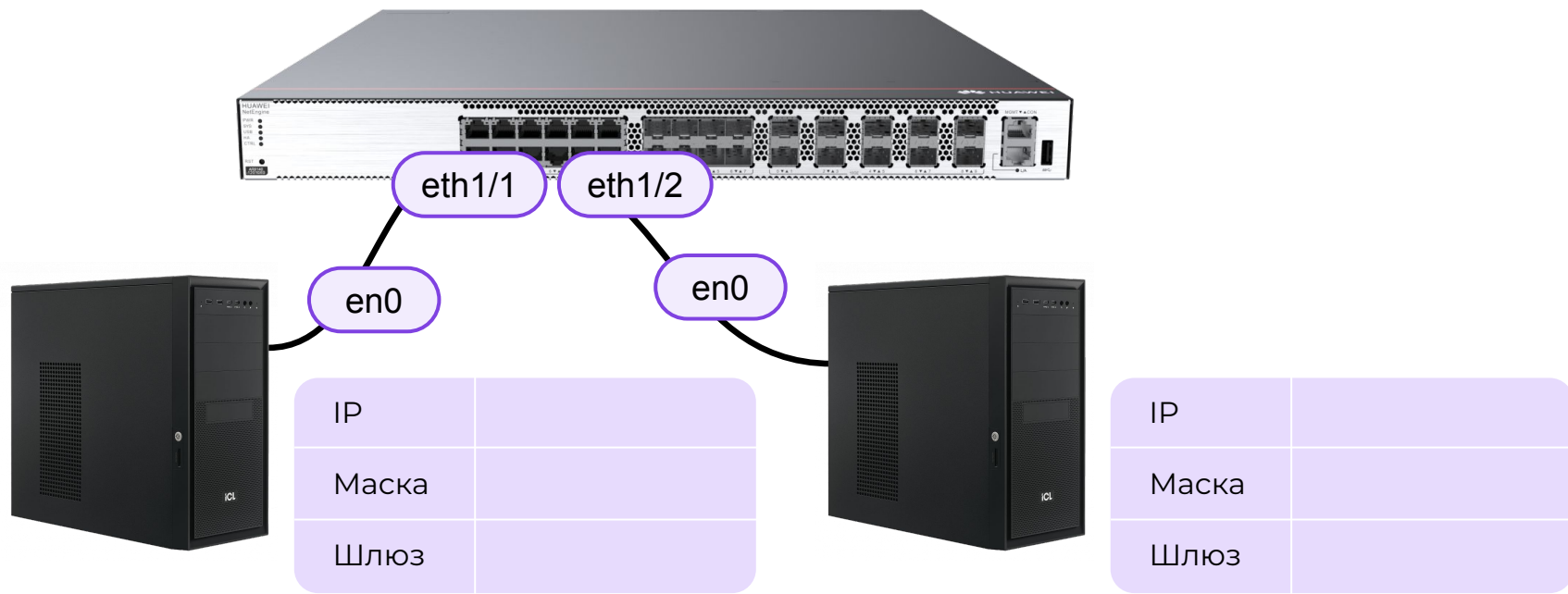
Подключение двух устройств

точка банк

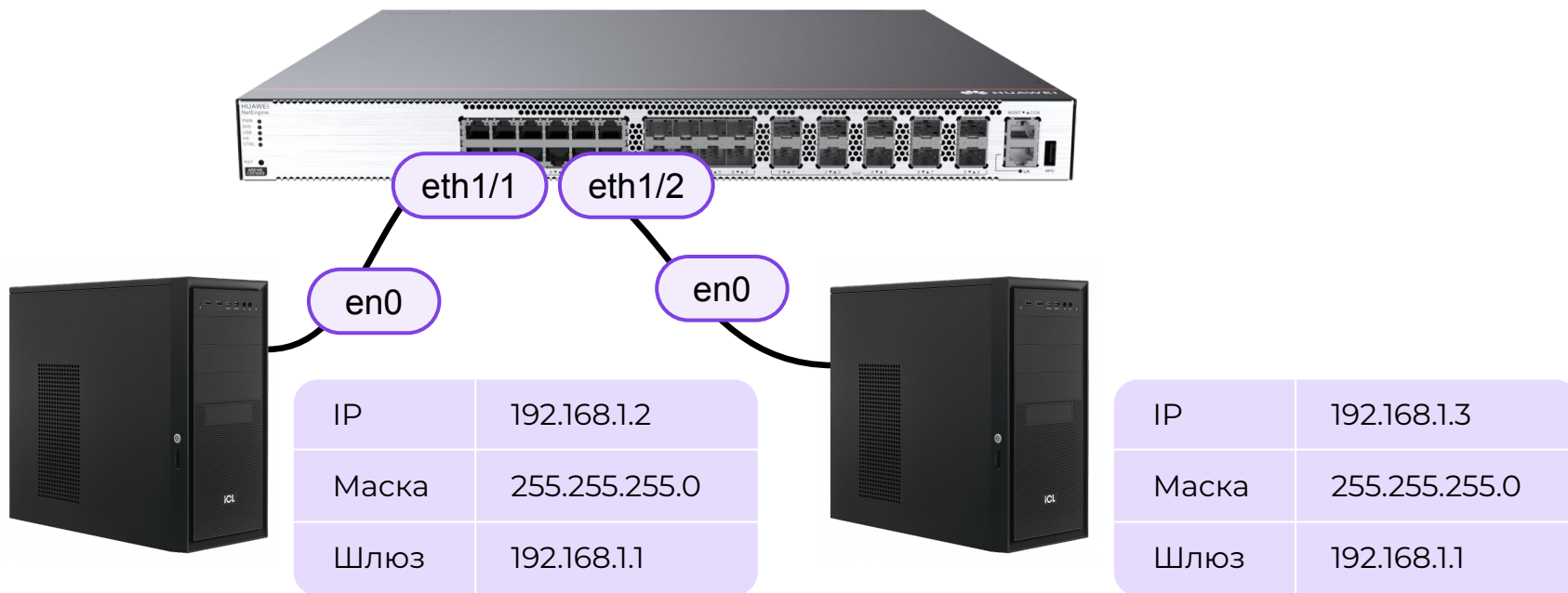


Подключение двух устройств

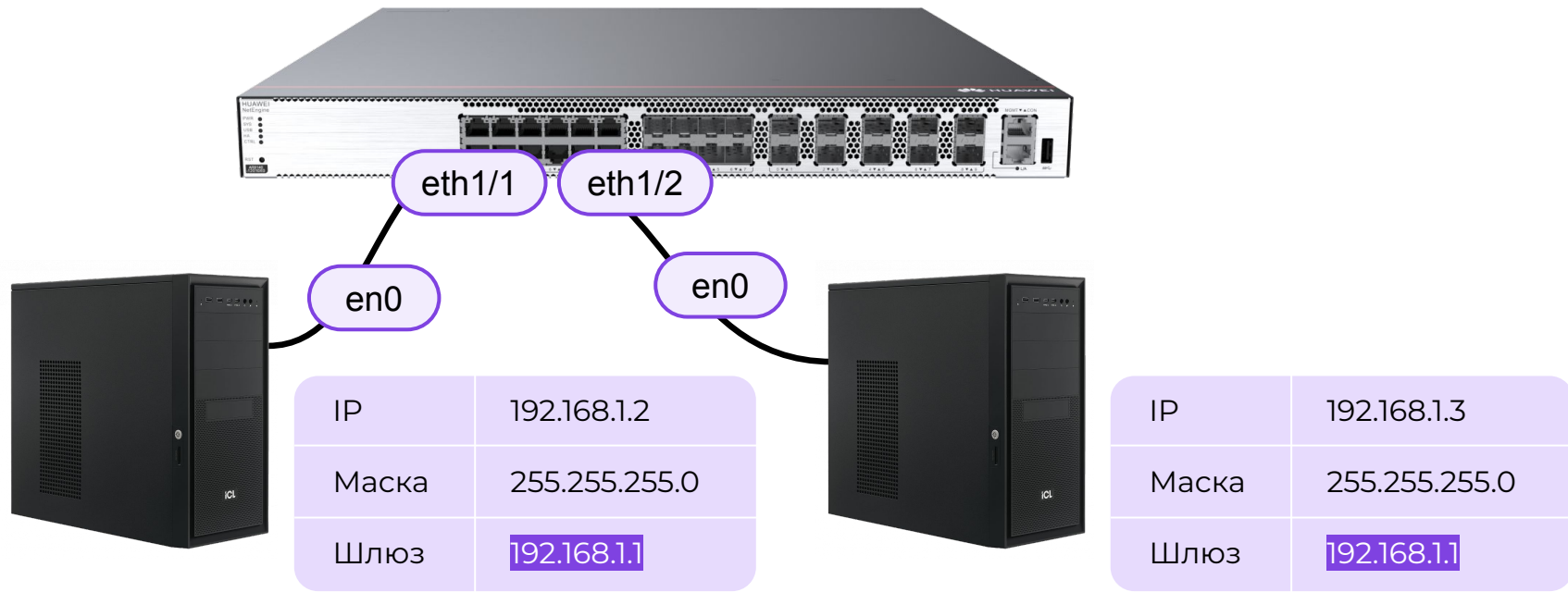
точка банк



Подключение двух устройств



Подключение двух устройств



Как это выглядит в GO?

точка банк

Как это выглядит в GO?

```
1.  interfaces, err := net.Interfaces()
2.  if err != nil {
3.      panic(err)
4.  }
5.  for _, iface := range interfaces {
6.      fmt.Printf("Interface: %s\n", iface.Name)
7.      fmt.Printf("  MAC: %s\n", iface.HardwareAddr)
8.      fmt.Printf("  Flags: %s\n", iface.Flags)
9.      // Получить IP-адреса интерфейса
10.     addrs, err := iface.Addrs()
11.     if err != nil {
12.         continue
13.     }
14.     for _, addr := range addrs {
15.         fmt.Printf("  Address: %s\n", addr)
16.     }
17.     fmt.Println()
18. }
```

Как это выглядит в GO?

```
1.  interfaces, err := net.Interfaces()
2.  if err != nil {
3.      panic(err)
4.  }
5.  for _, iface := range interfaces {
6.      fmt.Printf("Interface: %s\n", iface.Name)
7.      fmt.Printf("  MAC: %s\n", iface.HardwareAddr)
8.      fmt.Printf("  Flags: %s\n", iface.Flags)
9.      // Получить IP-адреса интерфейса
10.     addrs, err := iface.Addrs()
11.     if err != nil {
12.         continue
13.     }
14.     for _, addr := range addrs {
15.         fmt.Printf("    Address: %s\n", addr)
16.     }
17.     fmt.Println()
18. }
```

Как это выглядит в GO?

```
1. interfaces, err := net.Interfaces()
2. if err != nil {
3.     panic(err)
4. }
5. for _, iface := range interfaces {
6.     fmt.Printf("Interface: %s\n", iface.Name)
7.     fmt.Printf("  MAC: %s\n", iface.HardwareAddr)
8.     fmt.Printf("  Flags: %s\n", iface.Flags)
9.     // Получить IP-адреса интерфейса
10.    addrs, err := iface.Addrs()
11.    if err != nil {
12.        continue
13.    }
14.    for _, addr := range addrs {
15.        fmt.Printf("  Address: %s\n", addr)
16.    }
17.    fmt.Println()
18. }
```

Как это выглядит в GO?

```
1.  interfaces, err := net.Interfaces()
2.  if err != nil {
3.      panic(err)
4.  }
5.  for _, iface := range interfaces {
6.      fmt.Printf("Interface: %s\n", iface.Name)
7.      fmt.Printf("  MAC: %s\n", iface.HardwareAddr)
8.      fmt.Printf("  Flags: %s\n", iface.Flags)
9.      // Получить IP-адреса интерфейса
10.     addrs, err := iface.Addrs()
11.     if err != nil {
12.         continue
13.     }
14.     for _, addr := range addrs {
15.         fmt.Printf("  Address: %s\n", addr)
16.     }
17.     fmt.Println()
18. }
```

Как это выглядит в GO?

```
1. interfaces, err := net.Interfaces()
2. if err != nil {
3.     panic(err)
4. }
5. for _, iface := range interfaces {
6.     fmt.Printf("Interface: %s\n", iface.Name)
7.     fmt.Printf("  MAC: %s\n", iface.HardwareAddr)
8.     fmt.Printf("  Flags: %s\n", iface.Flags)
9.     // Получить IP-адреса интерфейса
10.    addrs, err := iface.Addrs()
11.    if err != nil {
12.        continue
13.    }
14.    for _, addr := range addrs {
15.        fmt.Printf("    Address: %s\n", addr)
16.    }
17.    fmt.Println()
18. }
```

```
1. Interface: en0
2.   MAC: ed:54:80:f3:d4:47
3.   Flags:
   up|broadcast|multicast|running
4.   Address: 192.168.1.2/24
```

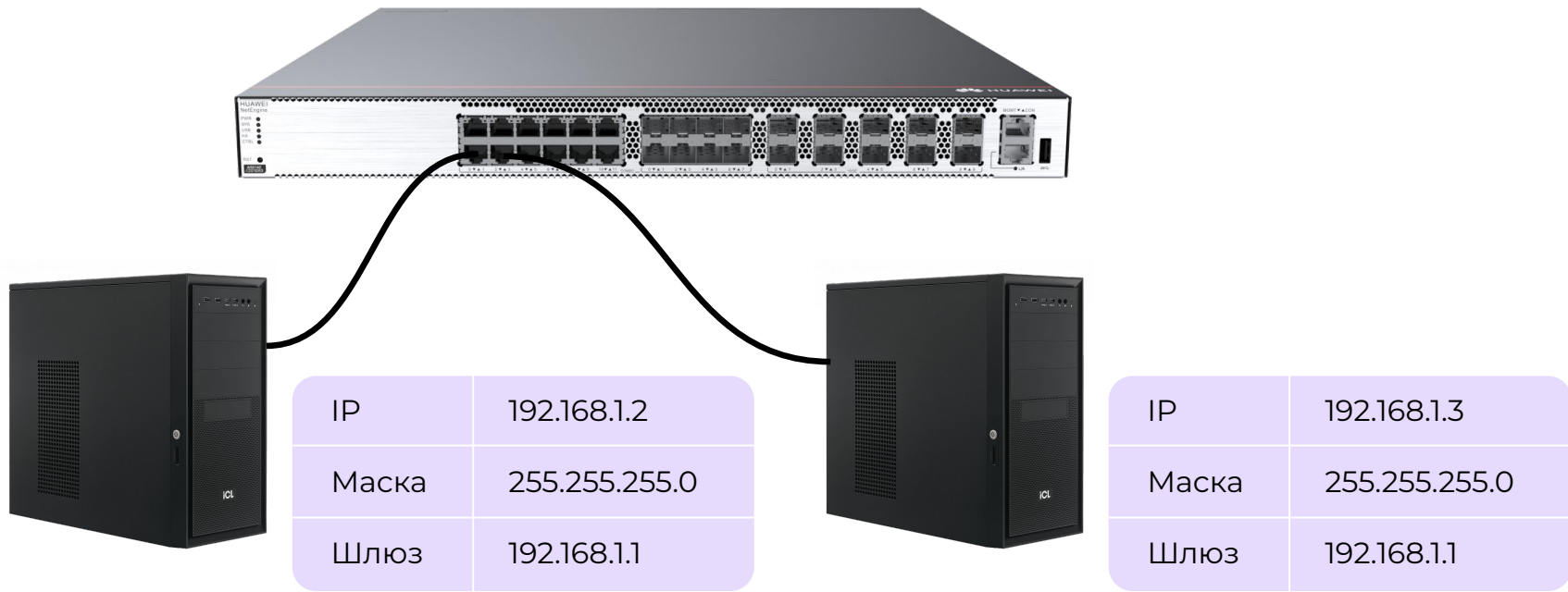
Как это выглядит в GO?

```
1. interfaces, err := net.Interfaces()
2. if err != nil {
3.     panic(err)
4. }
5. for _, iface := range interfaces {
6.     fmt.Printf("Interface: %s\n", iface.Name)
7.     fmt.Printf("  MAC: %s\n", iface.HardwareAddr)
8.     fmt.Printf("  Flags: %s\n", iface.Flags)
9.     // Получить IP-адреса интерфейса
10.    addrs, err := iface.Addrs()
11.    if err != nil {
12.        continue
13.    }
14.    for _, addr := range addrs {
15.        fmt.Printf("    Address: %s\n", addr)
16.    }
17.    fmt.Println()
18. }
```

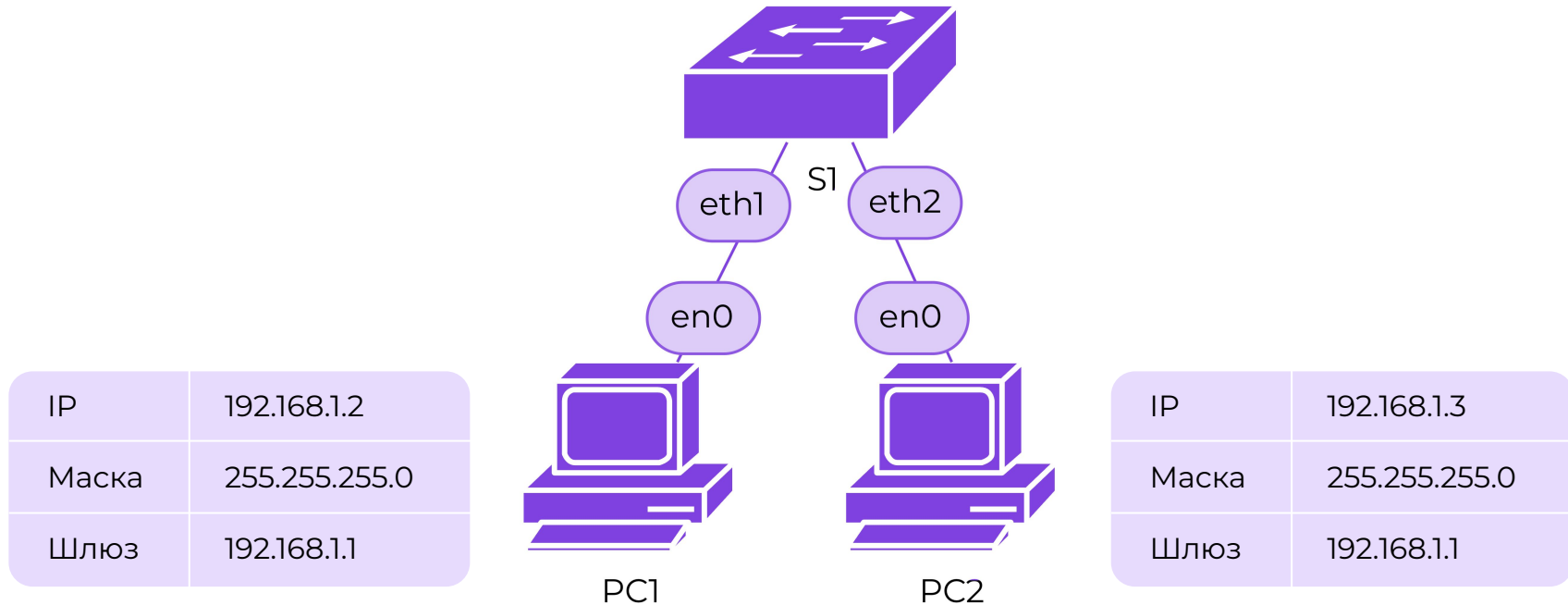
```
1. Interface: en0
2.   MAC: ed:54:80:f3:d4:47
3.   Flags:
   up|broadcast|multicast|running
4.   Address: 2001:db8:8714:3a90::64
5.   Address: 192.168.1.2/24
```

Схема была вот такая

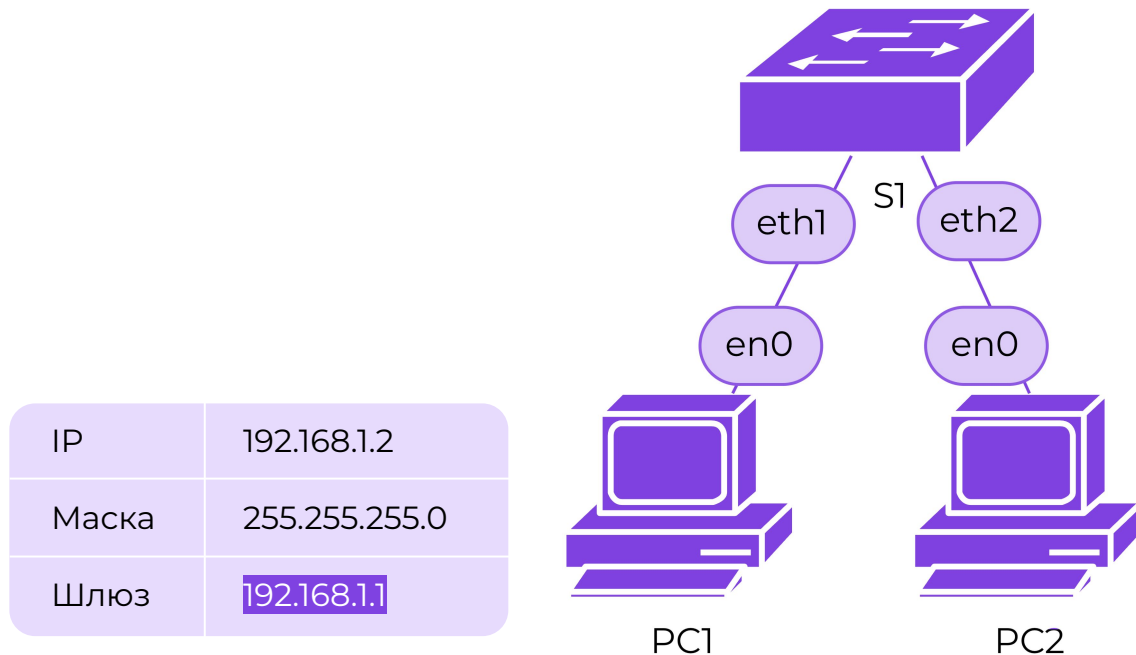
точка банк



А стала вот такая



А стала вот такая



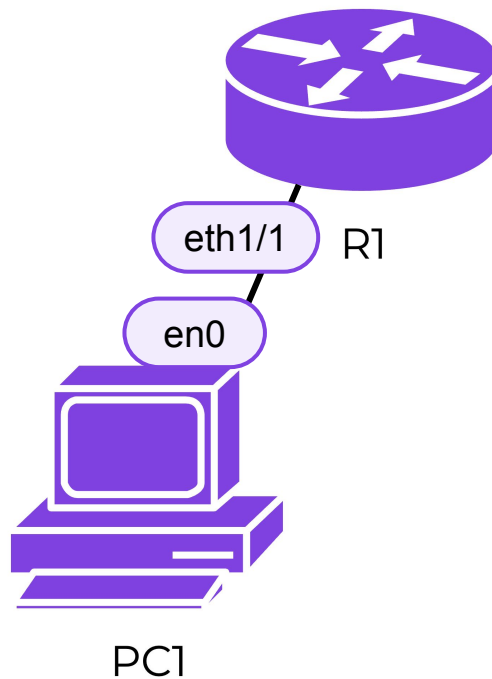
IP	192.168.1.2
Маска	255.255.255.0
Шлюз	192.168.1.1

IP	192.168.1.3
Маска	255.255.255.0
Шлюз	192.168.1.1

Автоматический IP

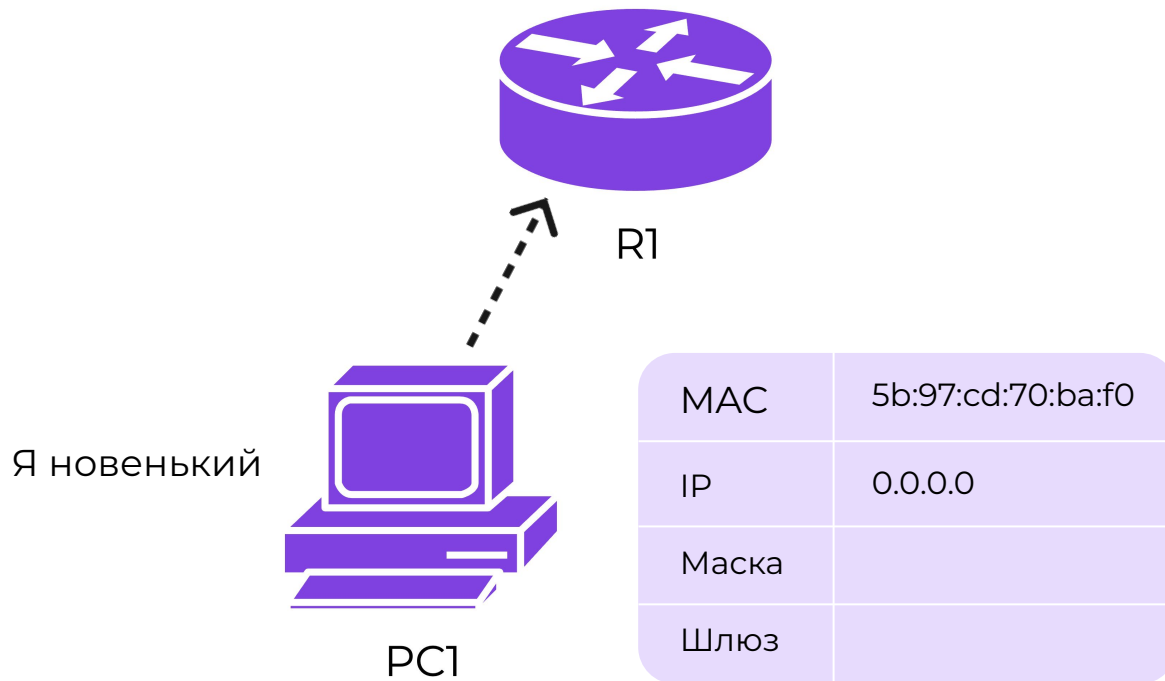
Dynamic Host Configuration Protocol

точка банк



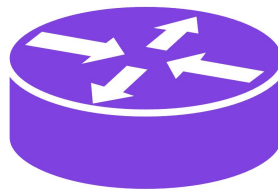
Dynamic Host Configuration Protocol

точка банк



Dynamic Host Configuration Protocol

IP	192.168.1.10
Маска	255.255.255.0
Шлюз	192.168.1.1
DNS	192.168.1.99



R1

Вот тебе адрес дома

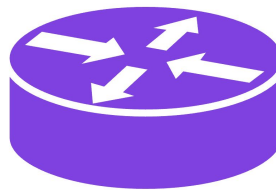


PC1

MAC	5b:97:cd:70:ba:f0
IP	0.0.0.0
Маска	
Шлюз	

Dynamic Host Configuration Protocol

IP	MAC
192.168.1.10	5b:97:cd:70:ba:f0
192.168.1.9	b7:7b:85:6b:ec:96



R1

Ок, я буду жить
по этому адресу

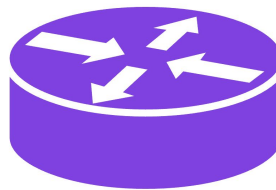


PC1

MAC	5b:97:cd:70:ba:f0
IP	192.168.1.10
Маска	255.255.255.0
Шлюз	192.168.1.1

Dynamic Host Configuration Protocol

IP	MAC
192.168.1.10	5b:97:cd:70:ba:f0
192.168.1.9	b7:7b:85:6b:ec:96



Ок, твой IP 192.168.1.10

R1



PC1

MAC	5b:97:cd:70:ba:f0
IP	192.168.1.10
Маска	255.255.255.0
Шлюз	192.168.1.1

DHCP и GO

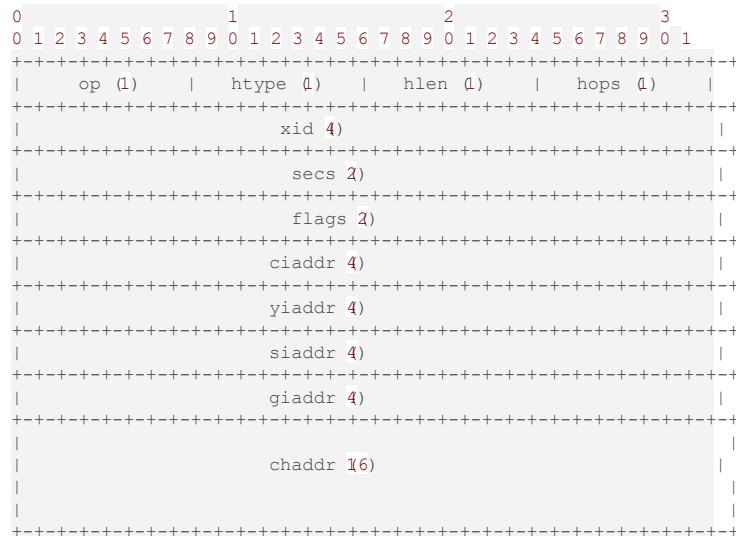
```
1. client, err := dhcp4client.New()
2. if err != nil {
3.     log.Fatal(err)
4. }
5. defer client.Close()
6. success, p, err := client.Request()
7. fmt.Printf("Success:%v\n", success)
8. fmt.Printf("Hardware Type: %d\n", p.HType())
9. fmt.Printf("MAC Length: %d\n", p.HLen())
10. fmt.Printf("Transaction ID: %x\n", p.XId())
11. fmt.Printf("DHCP Server: %s\n", p.SIAddr())
12. fmt.Printf("Your IP: %s\n", p.YIAddr())
13. fmt.Printf("Client MAC: %s\n", p.CHAddr())
14. fmt.Printf("Gateway: %s\n", p.GIAddr())
```

DHCP и GO

```
1. client, err := dhcp4client.New()
2. if err != nil {
3.     log.Fatal(err)
4. }
5. defer client.Close()
6. success, p, err := client.Request()
7. fmt.Printf("Success:%v\n", success)
8. fmt.Printf("Hardware Type: %d\n", p.HType())
9. fmt.Printf("MAC Length: %d\n", p.HLen())
10. fmt.Printf("Transaction ID: %x\n", p.XId())
11. fmt.Printf("DHCP Server: %s\n", p.SIAddr())
12. fmt.Printf("Your IP: %s\n", p.YIAddr())
13. fmt.Printf("Client MAC: %s\n", p.CHAddr())
14. fmt.Printf("Gateway: %s\n", p.GIAddr())
```

DHCP и GO

```
1. client, err := dhcp4client.New()
2. if err != nil {
3.     log.Fatal(err)
4. }
5. defer client.Close()
6. success, p, err := client.Request()
7. fmt.Printf("Success:%v\n", success)
8. fmt.Printf("Hardware Type: %d\n", p.HType())
9. fmt.Printf("MAC Length: %d\n", p.HLen())
10. fmt.Printf("Transaction ID: %x\n", p.XId())
11. fmt.Printf("DHCP Server: %s\n", p.SIAddr())
12. fmt.Printf("Your IP: %s\n", p.YIAddr())
13. fmt.Printf("Client MAC: %s\n", p.CHAddr())
14. fmt.Printf("Gateway: %s\n", p.GIAddr())
```



DHCP и GO

```
1. client, err := dhcp4client.New()
2. if err != nil {
3.     log.Fatal(err)
4. }
5. defer client.Close()
6. success, p, err := client.Request()
7. fmt.Printf("Success:%v\n", success)
8. fmt.Printf("Hardware Type: %d\n", p.HType())
9. fmt.Printf("MAC Length: %d\n", p.HLen())
10. fmt.Printf("Transaction ID: %x\n", p.XId())
11. fmt.Printf("DHCP Server: %s\n", p.SIAddr())
12. fmt.Printf("Your IP: %s\n", p.YIAddr())
13. fmt.Printf("Client MAC: %s\n", p.CHAddr())
14. fmt.Printf("Gateway: %s\n", p.GIAddr())
```

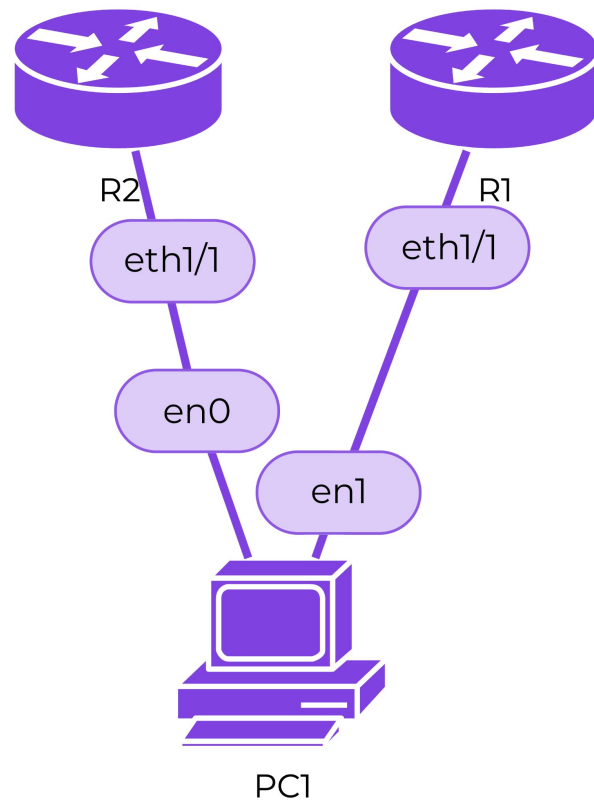
DHCP и GO

```
1. client, err := dhcp4client.New()
2. if err != nil {
3.     log.Fatal(err)
4. }
5. defer client.Close()
6. success, p, err := client.Request()
7. fmt.Printf("Success:%v\n", success)
8. fmt.Printf("Hardware Type: %d\n", p.HType())
9. fmt.Printf("MAC Length: %d\n", p.HLen())
10. fmt.Printf("Transaction ID: %x\n", p.XId())
11. fmt.Printf("DHCP Server: %s\n", p.SIAddr())
12. fmt.Printf("Your IP: %s\n", p.YIAddr())
13. fmt.Printf("Client MAC: %s\n", p.CHAddr())
14. fmt.Printf("Gateway: %s\n", p.GIAddr())
```

```
1. Success: true
2. Packet length: 576 bytes
3. Is valid: true
4. OpCode: 2 (1=Request, 2=Reply)
5. Hardware Type: 1
6. MAC Length: 6
7. Hops: 0
8. Transaction ID: 833651fa
9. DHCP Server: 192.168.1.1
10. Your IP: 192.168.1.10
11. Client IP: 0.0.0.0
12. Gateway: 192.168.1.1
13. Client MAC: 5b:97:cd:70:ba:f0
```

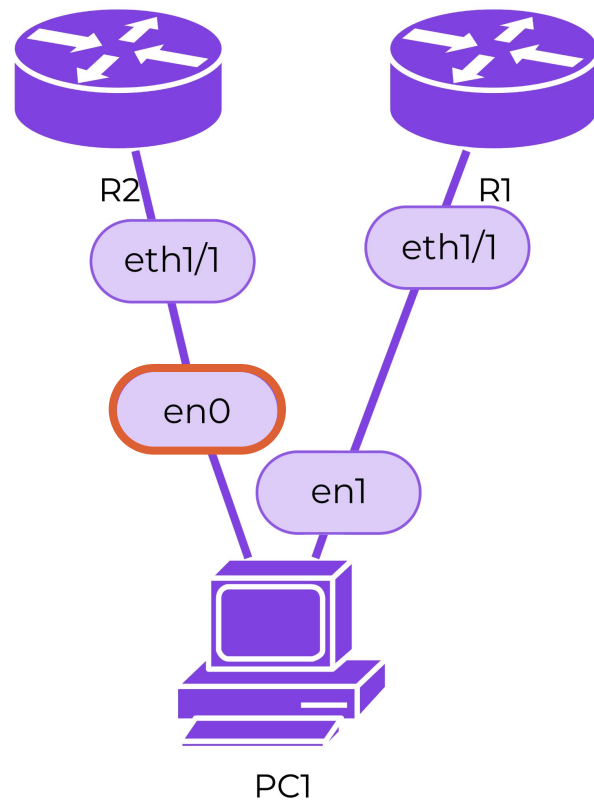
А если есть порты?

точка банк



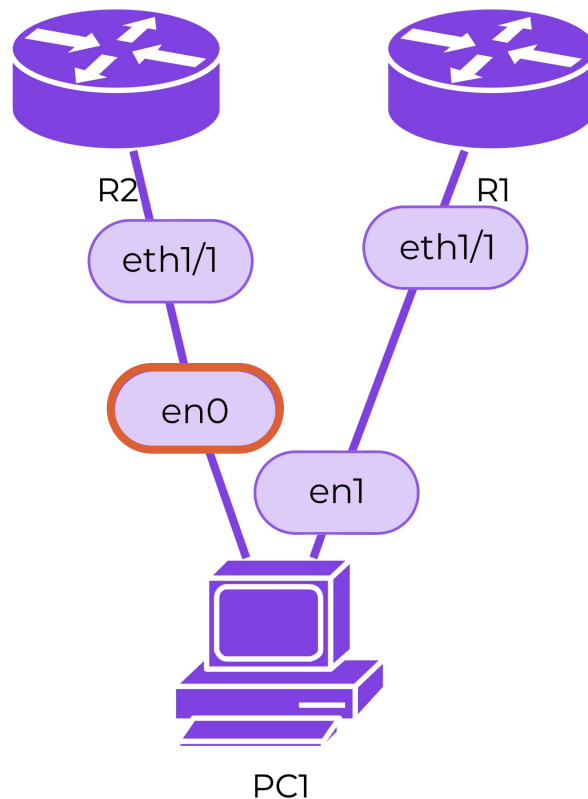
А если есть порты?

точка банк



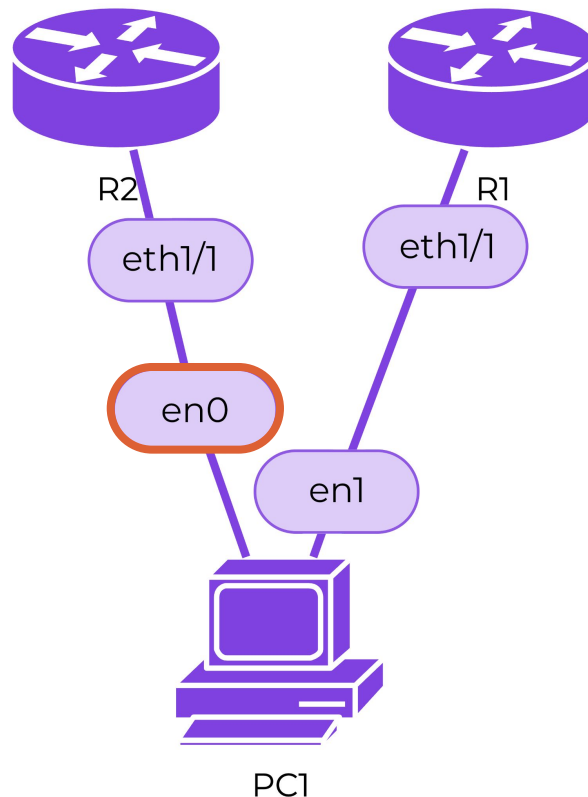
А если есть порты?

```
1.  iface, err := net.InterfaceByName( "en0")
2.  if err != nil {
3.    log.Fatal(err)
4.  }
5.  client, err := dhcp4client.New(
6.    dhcp4client.HardwareAddr(iface.HardwareAddr),
7.    dhcp4client.Timeout( 10),
8.  )
9.  if err != nil {
10.   log.Fatal(err)
11. }
12. defer client.Close()
13. success, packet, err := client.Request()
```



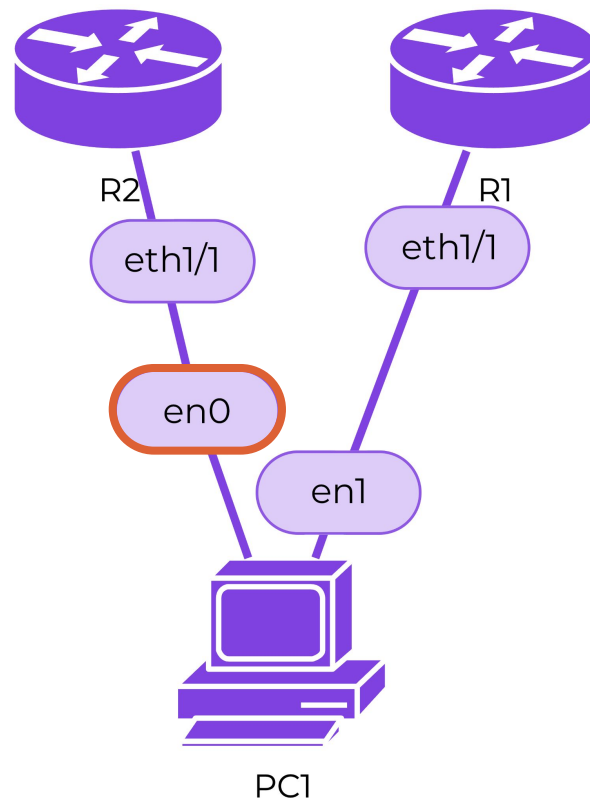
А если есть порты?

```
1. iface, err := net.InterfaceByName( "en0")
2. if err != nil {
3.     log.Fatal(err)
4. }
5. client, err := dhcp4client.New(
6.     dhcp4client.HardwareAddr(iface.HardwareAddr),
7.     dhcp4client.Timeout( 10),
8. )
9. if err != nil {
10.    log.Fatal(err)
11. }
12. defer client.Close()
13. success, packet, err := client.Request()
```



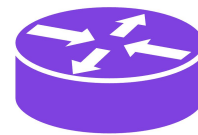
А если есть порты?

```
1.  iface, err := net.InterfaceByName( "en0")
2.  if err != nil {
3.    log.Fatal(err)
4.  }
5.  client, err := dhcp4client.New(
6.    dhcp4client.HardwareAddr(iface.HardwareAddr),
7.    dhcp4client.Timeout( 10),
8.  )
9.  if err != nil {
10.    log.Fatal(err)
11.  }
12.  defer client.Close()
13.  success, packet, err := client.Request()
```



Есть даже сервер!!!

точка банк



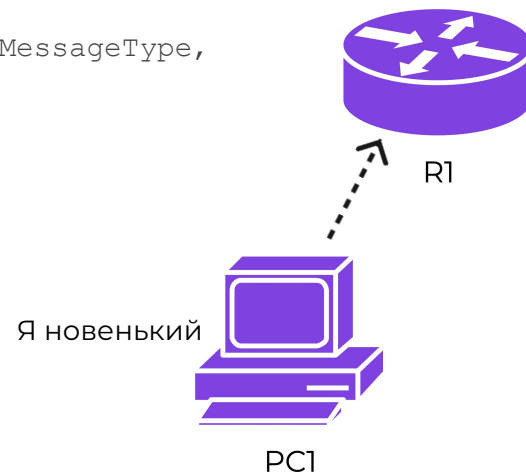
R1



PC1

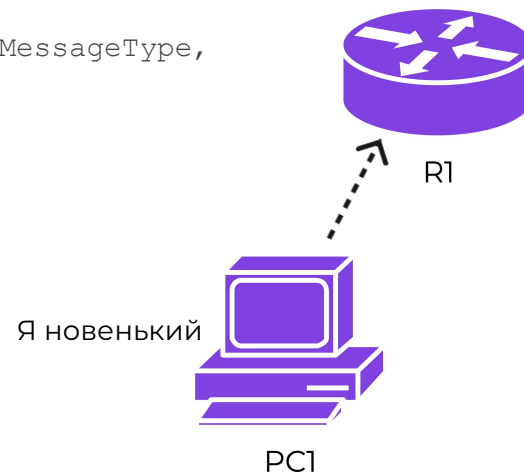
Есть даже сервер!!!

```
1. import dhcp "github.com/krolaw/dhcp4"
2. func (h *DHCPHandler) ServeDHCP(p dhcp.Packet, msgType dhcp.MessageType,
   options dhcp.Options) (d dhcp.Packet) {
3. case dhcp.Discover:
4.     free, nic := -1, p.CHAddr().String()
5.     for i, v := range h.leases {
6.         if v.nic == nic {
7.             free = i
8.             goto reply
9.         }
10.    }
11.    if free = h.freeLease(); free == -1 {
12.        return
13.    }
14. reply:
15.    return dhcp.ReplyPacket(p, dhcp.Offer, h.ip, dhcp.IPAdd(h.start, free),
        h.leaseDuration,
        h.options.SelectOrderOrAll(options[dhcp.OptionParameterRequestList]))
```



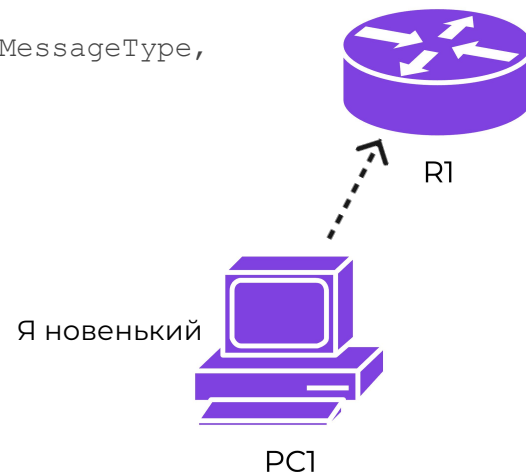
Есть даже сервер!!!

```
1. import dhcp "github.com/krolaw/dhcp4"
2. func (h *DHCPHandler) ServeDHCP(p dhcp.Packet, msgType dhcp.MessageType,
   options dhcp.Options) (d dhcp.Packet) {
3. case dhcp.Discover:
4.     free, nic := -1, p.CHAddr().String()
5.     for i, v := range h.leases {
6.         if v.nic == nic {
7.             free = i
8.             goto reply
9.         }
10.    }
11.    if free = h.freeLease(); free == -1 {
12.        return
13.    }
14. reply:
15.    return dhcp.ReplyPacket(p, dhcp.Offer, h.ip, dhcp.IPAdd(h.start, free),
        h.leaseDuration,
        h.options.SelectOrderOrAll(options[dhcp.OptionParameterRequestList]))
```



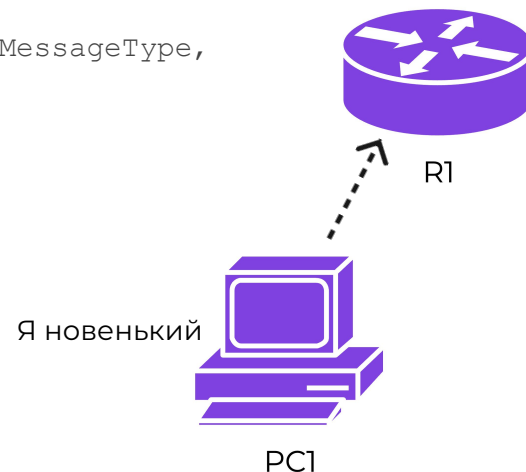
Есть даже сервер!!!

```
1. import dhcp "github.com/krolaw/dhcp4"
2. func (h *DHCPHandler) ServeDHCP(p dhcp.Packet, msgType dhcp.MessageType,
   options dhcp.Options) (d dhcp.Packet) {
3. case dhcp.Discover:
4.     free, nic := -1, p.CHAddr().String()
5.     for i, v := range h.leases {
6.         if v.nic == nic {
7.             free = i
8.             goto reply
9.         }
10.    }
11.    if free = h.freeLease(); free == -1 {
12.        return
13.    }
14. reply:
15.    return dhcp.ReplyPacket(p, dhcp.Offer, h.ip, dhcp.IPAdd(h.start, free),
        h.leaseDuration,
        h.options.SelectOrderOrAll(options[dhcp.OptionParameterRequestList]))
```



Есть даже сервер!!!

```
1. import dhcp "github.com/krolaw/dhcp4"
2. func (h *DHCPHandler) ServeDHCP(p dhcp.Packet, msgType dhcp.MessageType,
   options dhcp.Options) (d dhcp.Packet) {
3. case dhcp.Discover:
4.     free, nic := -1, p.CHAddr().String()
5.     for i, v := range h.leases {
6.         if v.nic == nic {
7.             free = i
8.             goto reply
9.         }
10.    }
11.    if free = h.freeLease(); free == -1 {
12.        return
13.    }
14. reply:
15.    return dhcp.ReplyPacket(p, dhcp.Offer, h.ip, dhcp.IPAdd(h.start, free),
   h.leaseDuration,
   h.options.SelectOrderOrAll(options[dhcp.OptionParameterRequestList]))
```



Зачем эти библиотеки?

Плюсы

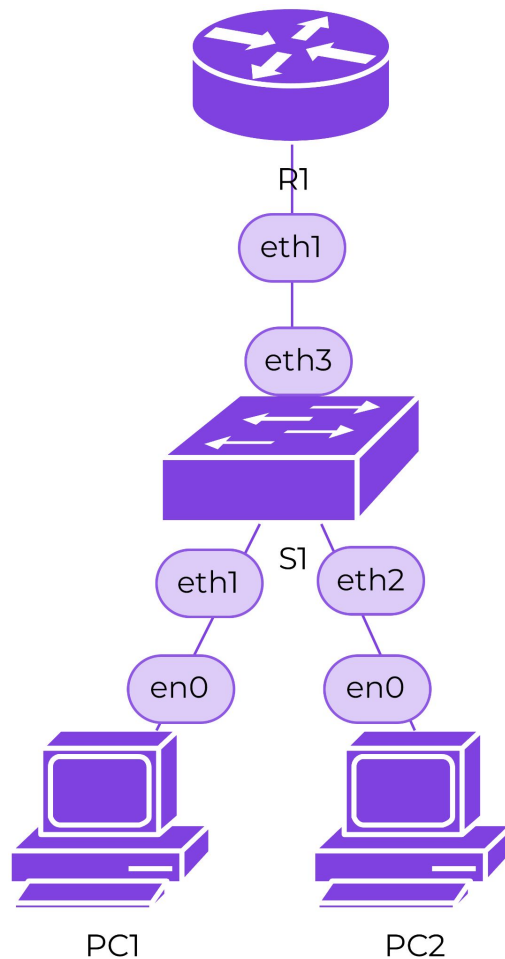
- 1 Одинаковый код и на любимом языке
- 2 Кросс-платформенность
- 3 Полный контроль и предсказуемость

Минусы

- 1 Производительность (overhead)
- 2 Библиотека работает на библиотеке, которая работает на библиотеке

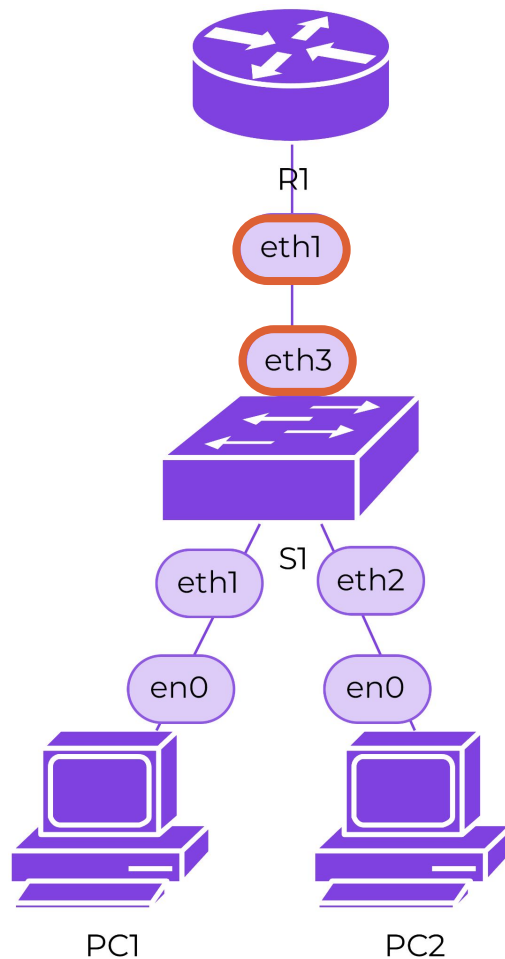
Правильная схема с DHCP

точка банк



Правильная схема с DHCP

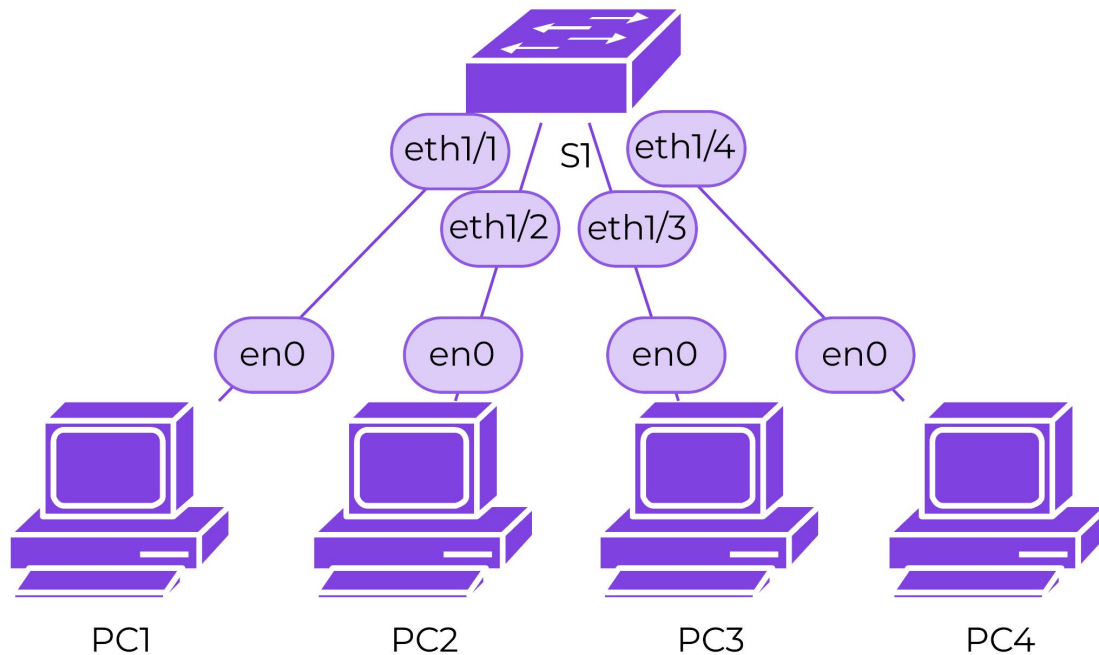
точка банк



Усложняем

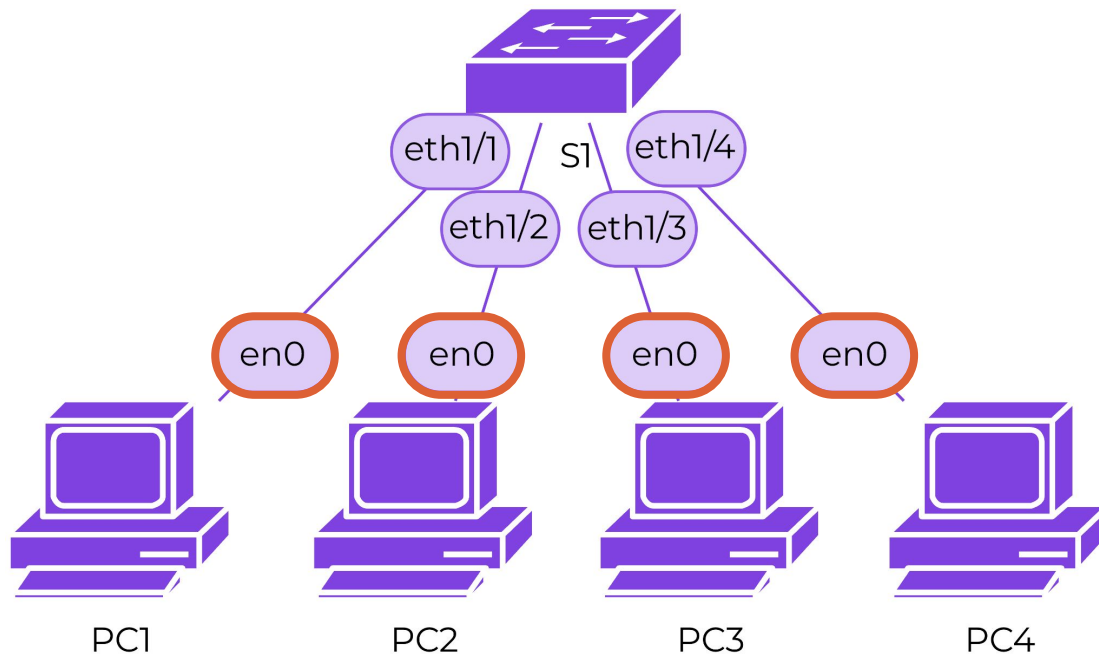
Исходная топология

точка банк



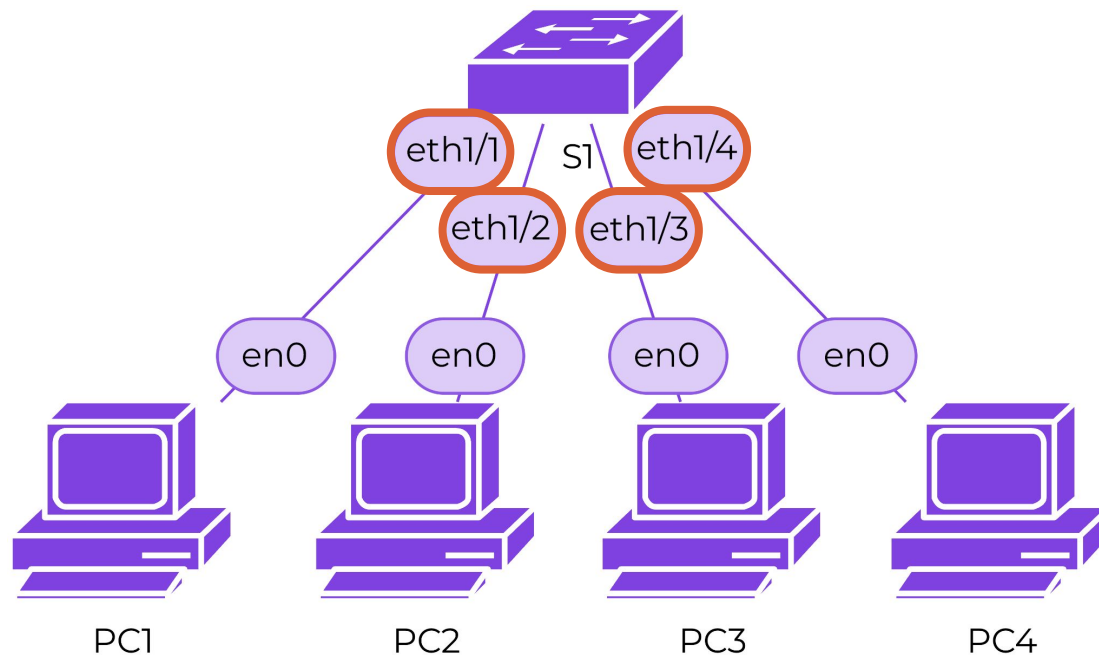
Исходная топология

точка банк



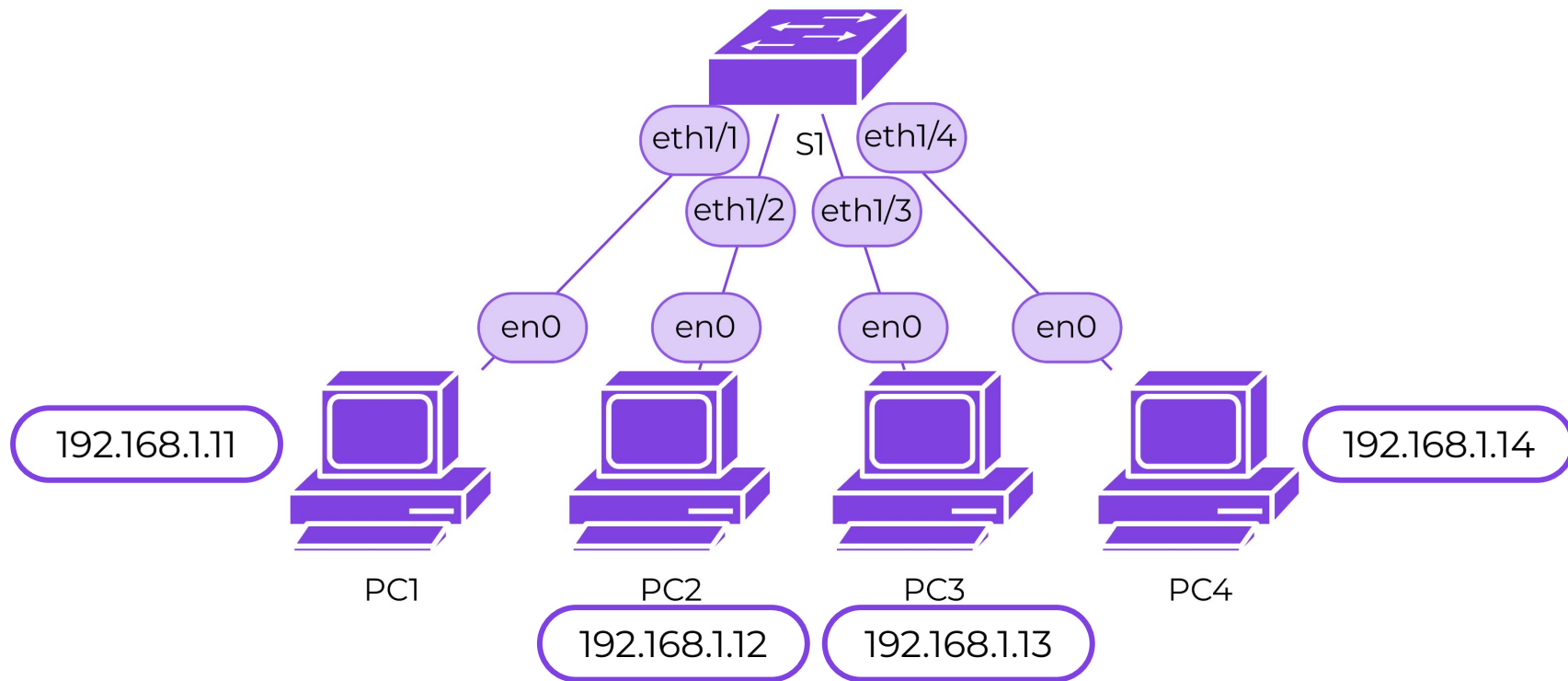
Исходная топология

точка банк



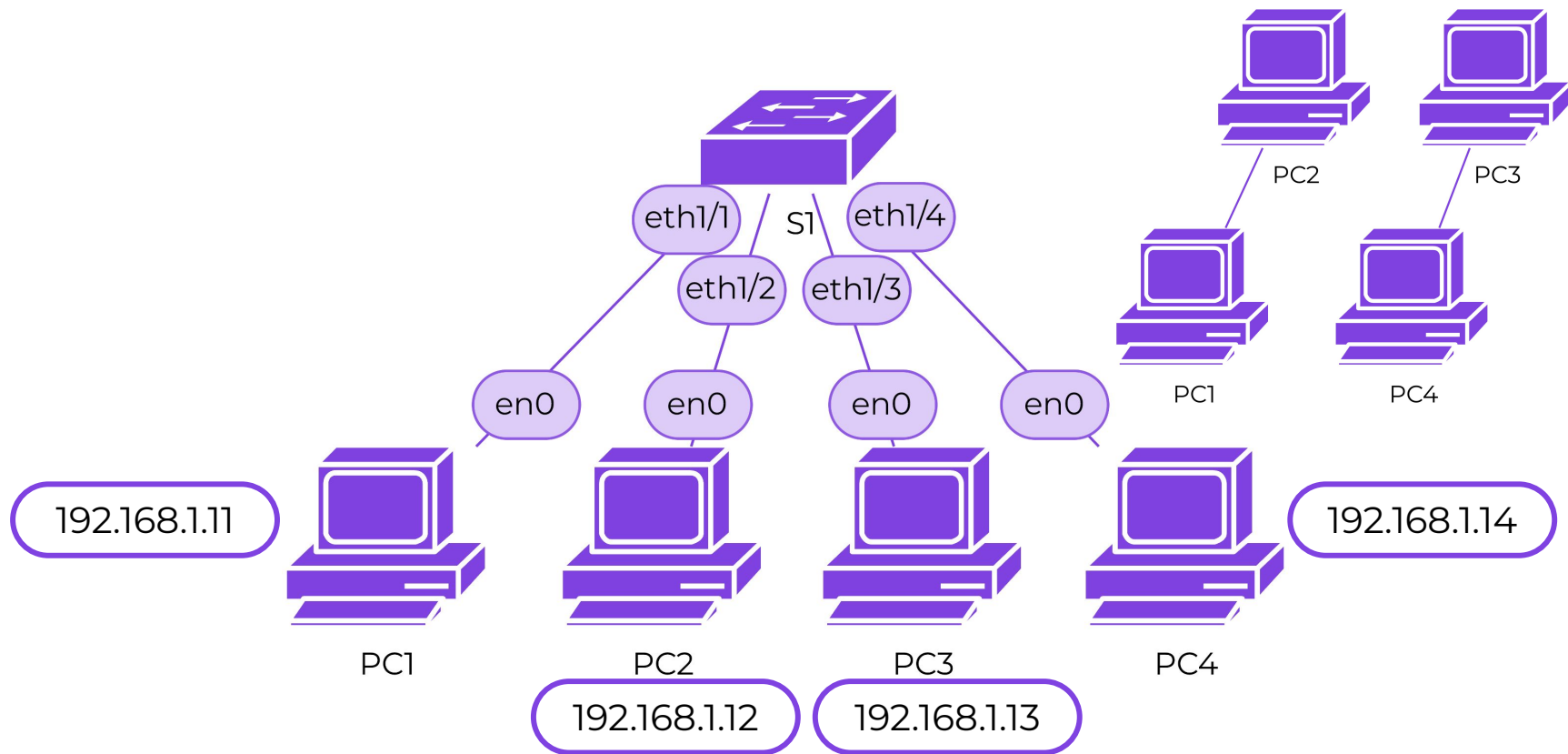
Исходная топология

точка банк

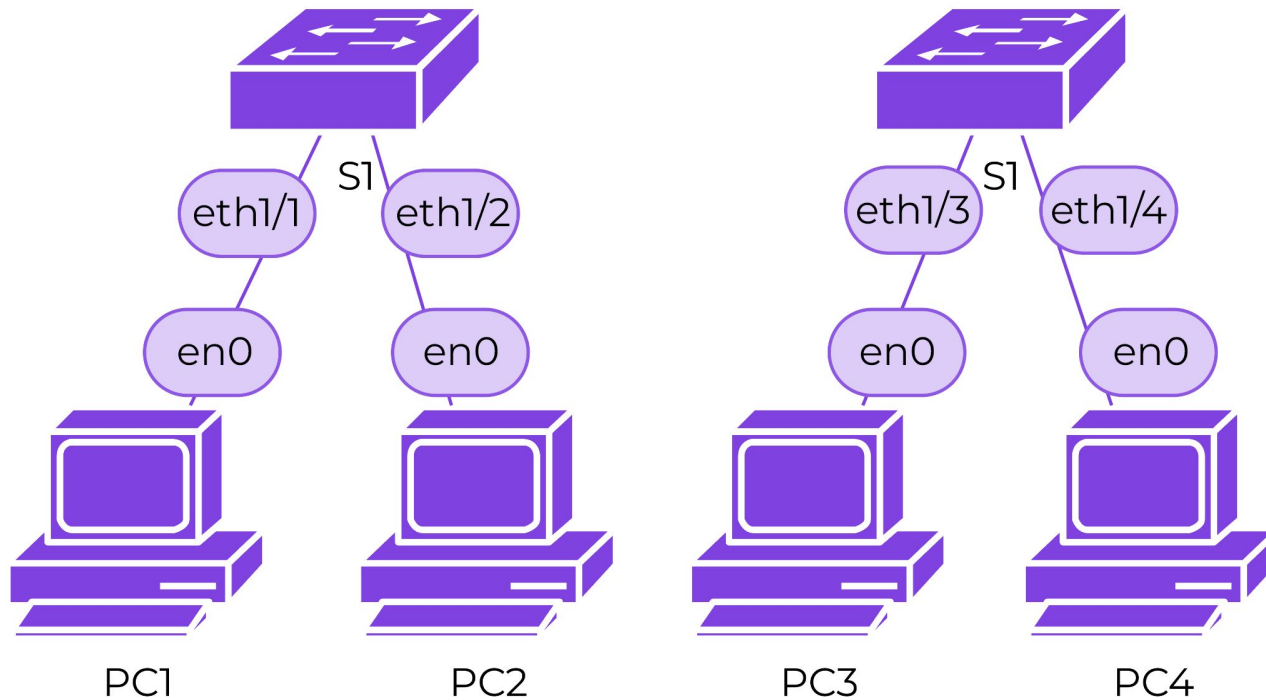


Исходная топология

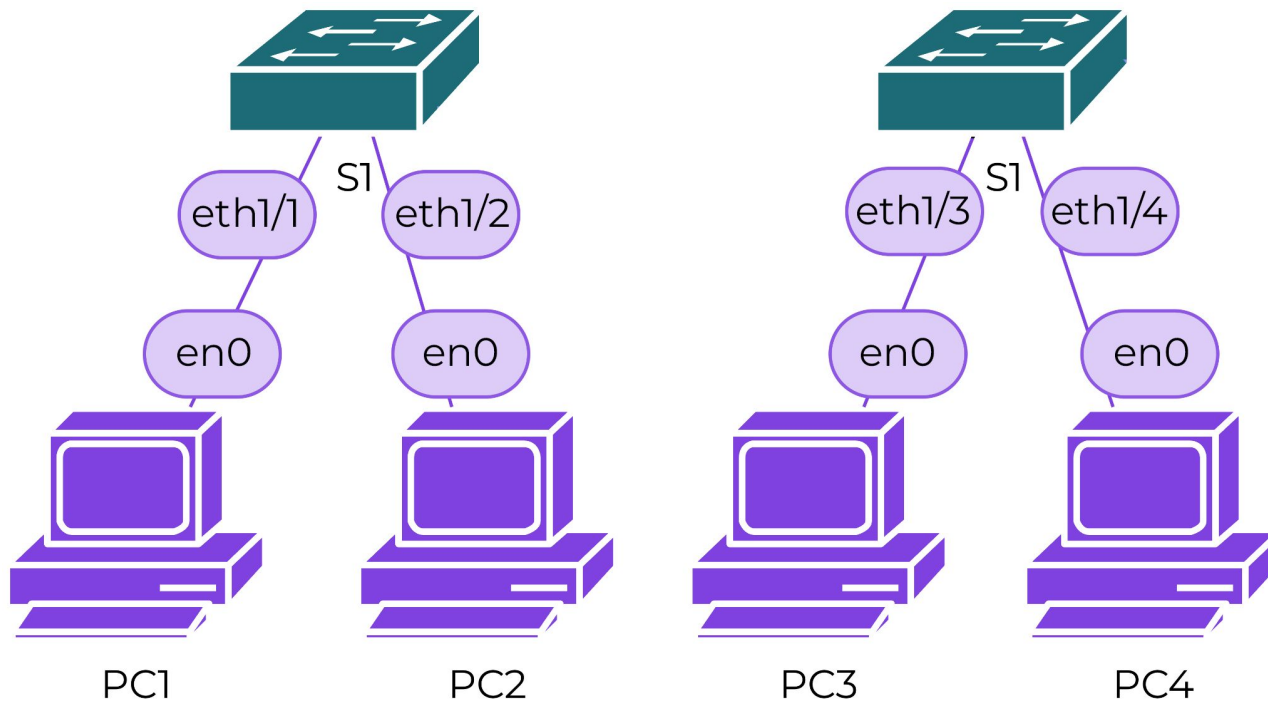
точка банк



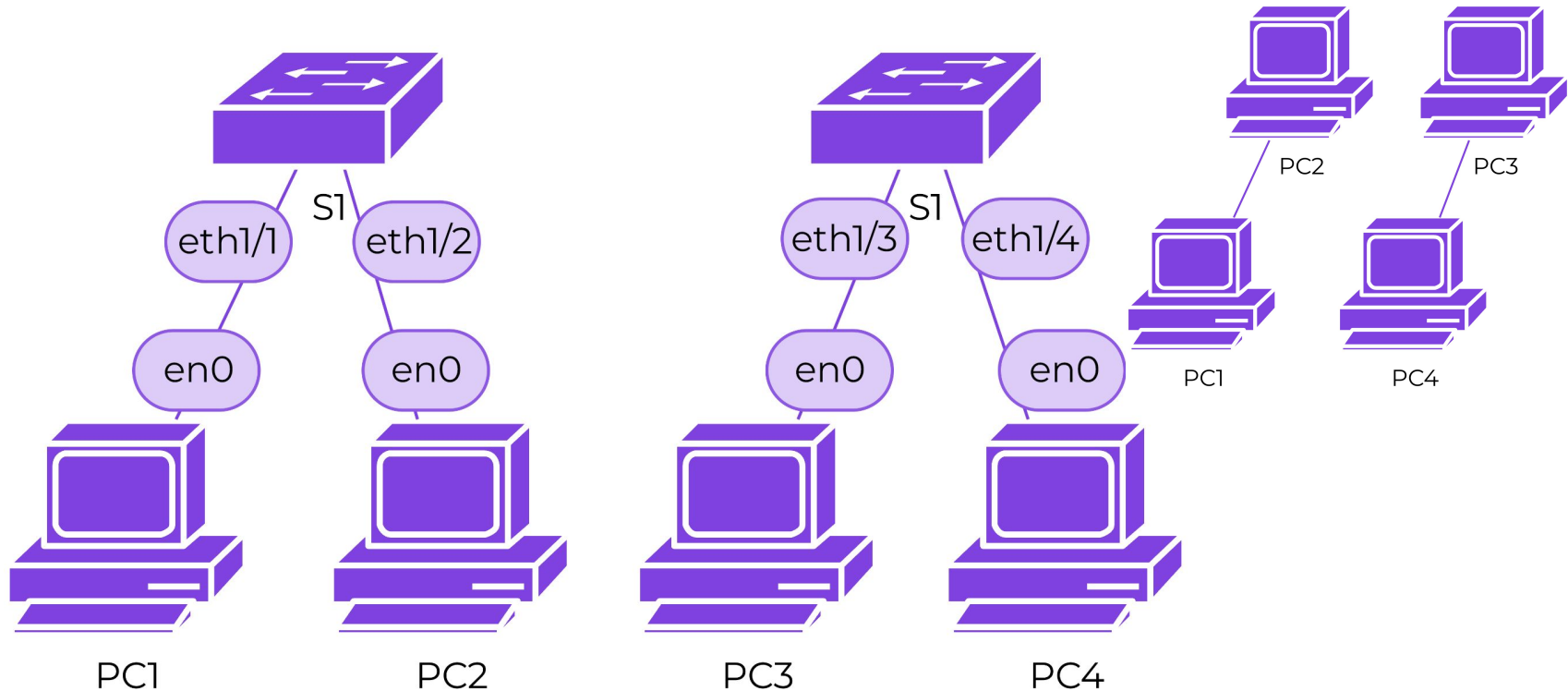
Хотим разделенную топологию



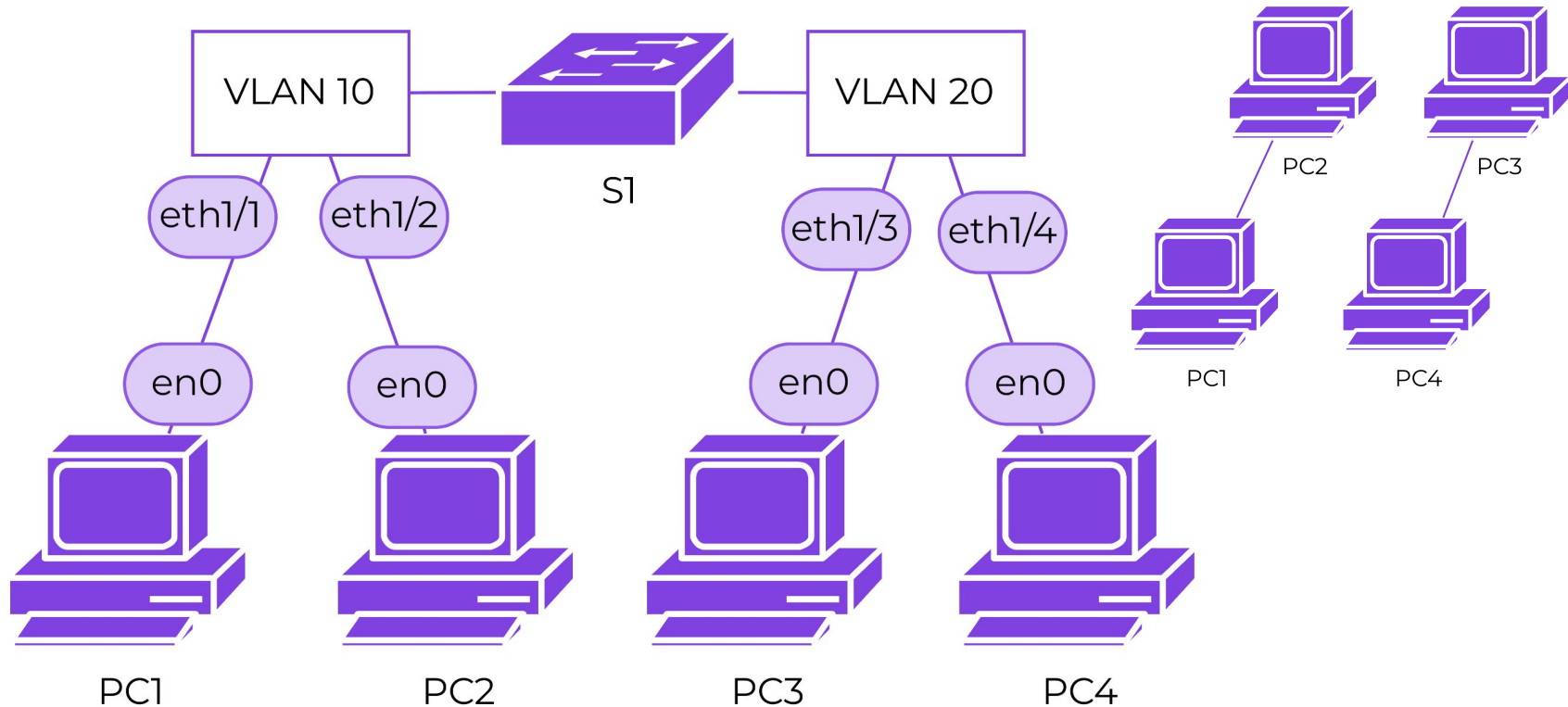
Хотим разделенную топологию



Хотим разделенную топологию



Хотим разделенную топологию



Пример настройки VLAN

```
1.  ! VLAN 10 - подсеть 1
2.  interface vlan10
3.  ip address 192.168.10.1 255.255.255.0
4.
5.  ! Настройка портов 1-2
6.  interface range eth1/1-2
7.  switchport access vlan 10
8.
9.  ! VLAN 20 - подсеть 2
10. interface vlan20
11. ip address 192.168.20.1 255.255.255.0
12.
13. ! Настройка портов 3-4
14. interface range eth1/3-4
15. switchport access vlan 20
```

Пример настройки VLAN

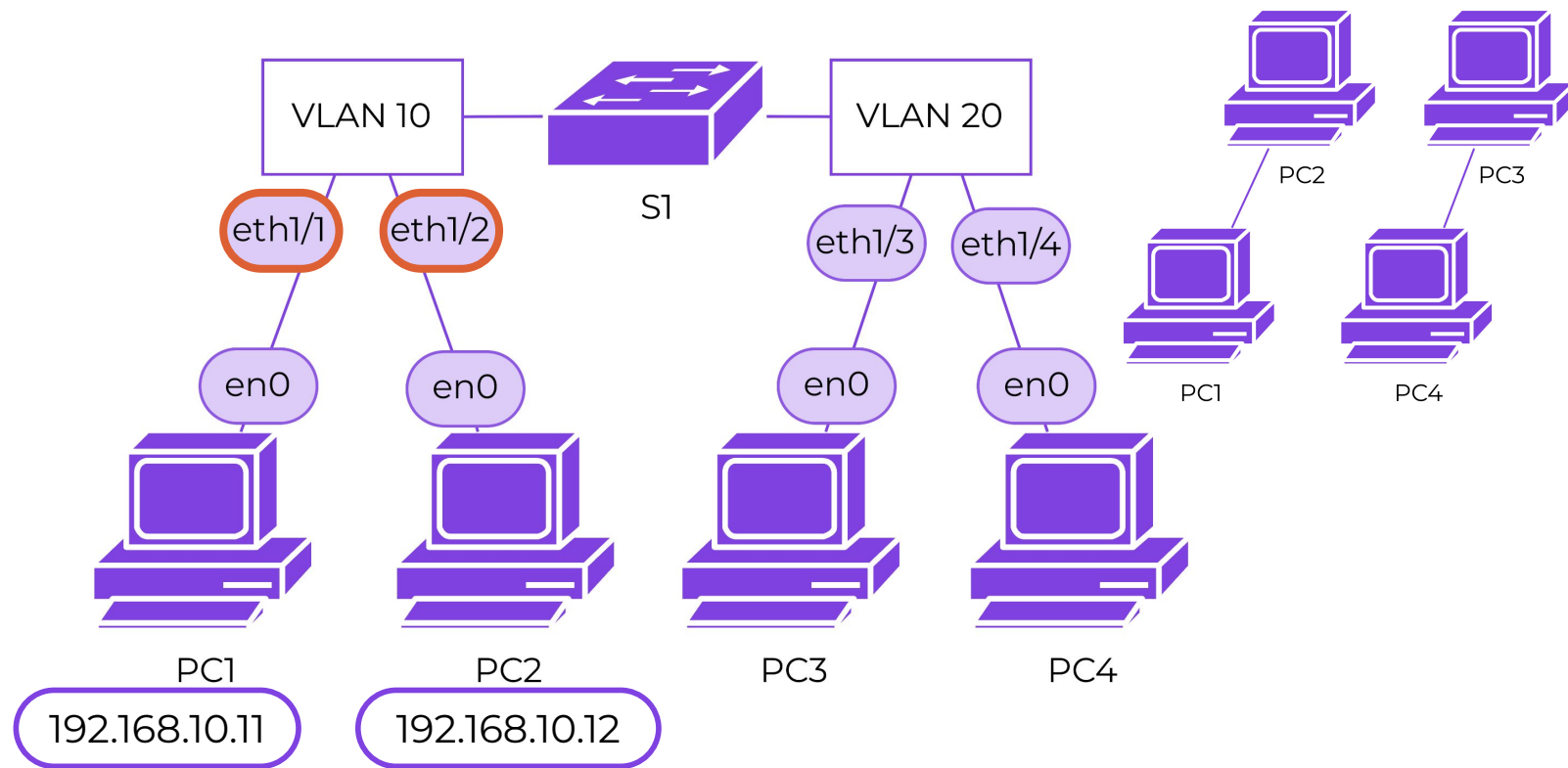
```
1.  ! VLAN 10 - подсеть 1
2.  interface vlan10
3.  ip address 192.168.10.1 255.255.255.0
4.
5.  ! Настройка портов 1-2
6.  interface range eth1/1-2
7.  switchport access vlan 10
8.
9.  ! VLAN 20 - подсеть 2
10. interface vlan20
11. ip address 192.168.20.1 255.255.255.0
12.
13. ! Настройка портов 3-4
14. interface range eth1/3-4
15. switchport access vlan 20
```

Пример настройки VLAN

```
1.  ! VLAN 10 - подсеть 1
2.  interface vlan10
3.  ip address 192.168.10.1 255.255.255.0
4.
5.  ! Настройка портов 1-2
6.  interface range eth1/1-2
7.  switchport access vlan 10
8.
9.  ! VLAN 20 - подсеть 2
10. interface vlan20
11. ip address 192.168.20.1 255.255.255.0
12.
13. ! Настройка портов 3-4
14. interface range eth1/3-4
15. switchport access vlan 20
```

Пример настройки VLAN

точка банк



Пример настройки VLAN

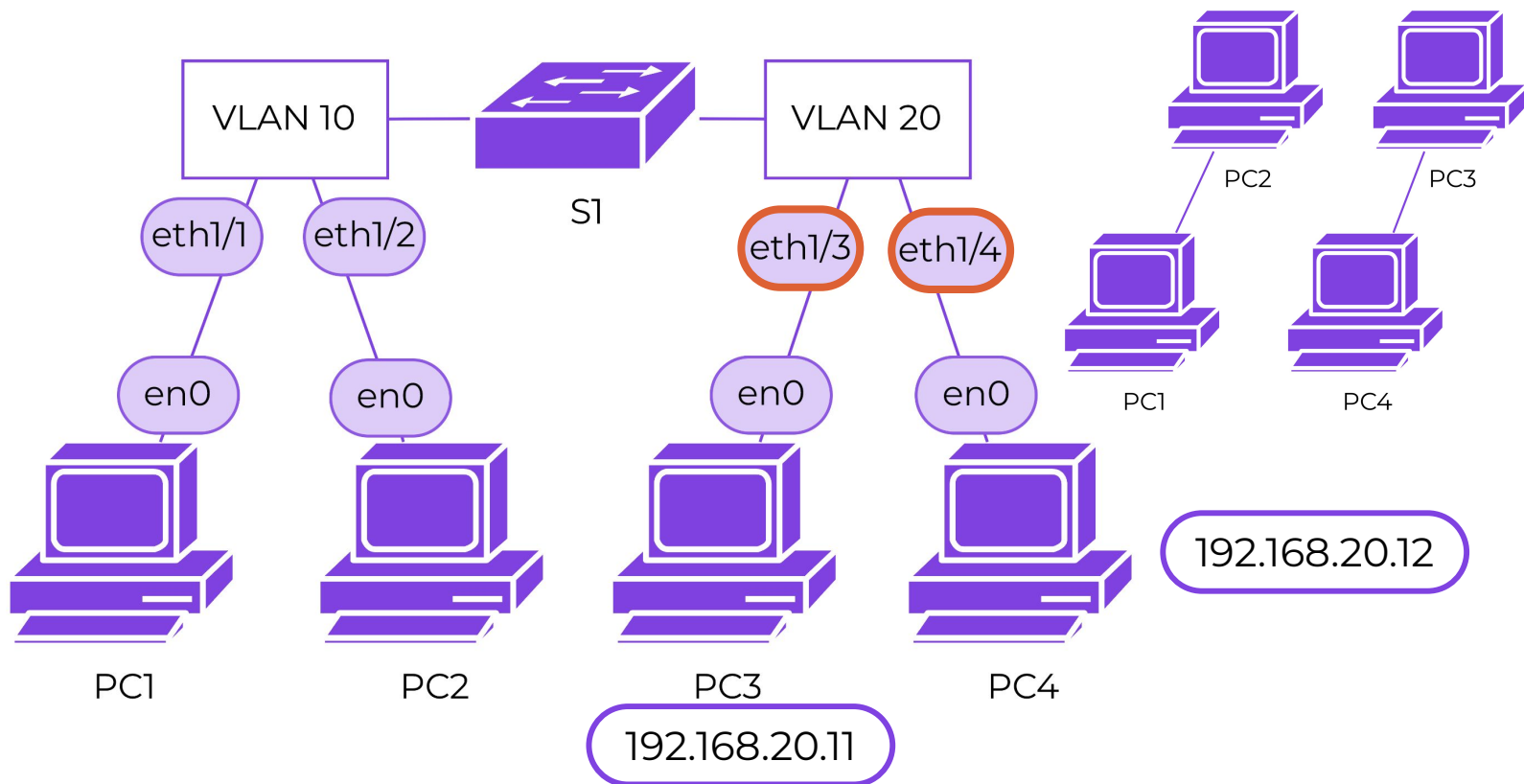
```
1.  ! VLAN 10 - подсеть 1
2.  interface vlan10
3.  ip address 192.168.10.1 255.255.255.0
4.
5.  ! Настройка портов 1-2
6.  interface range eth1/1-2
7.  switchport access vlan 10
8.
9.  ! VLAN 20 - подсеть 2
10. interface vlan20
11. ip address 192.168.20.1 255.255.255.0
12.
13. ! Настройка портов 3-4
14. interface range eth1/3-4
15. switchport access vlan 20
```

Пример настройки VLAN

```
1.  ! VLAN 10 - подсеть 1
2.  interface vlan10
3.  ip address 192.168.10.1 255.255.255.0
4.
5.  ! Настройка портов 1-2
6.  interface range eth1/1-2
7.  switchport access vlan 10
8.
9.  ! VLAN 20 - подсеть 2
10. interface vlan20
11. ip address 192.168.20.1 255.255.255.0
12.
13. ! Настройка портов 3-4
14. interface range eth1/3-4
15. switchport access vlan 20
```

Пример настройки VLAN

точка банк



Пример настройки VLAN на GO

```
1.  import "github.com/vishvananda/netlink"
2.  eth1, _ := netlink.LinkByName("eth1/1")
3.  vlan := &netlink.Vlan{
4.      LinkAttrs: netlink.LinkAttrs{
5.          Name:      "vlan.10",
6.          ParentIndex: eth1.Attrs().Index,
7.      },
8.      VlanId: 10,
9.  }
10. netlink.LinkAdd(vlan)
11. addr, _ := netlink.ParseAddr("192.168.10.1/24")
12. netlink.AddrAdd(vlan, addr)
13. netlink.LinkSetUp(vlan)
```

```
1.  ! VLAN 10 - подсеть 1
2.  interface vlan10
3.  ip address 192.168.10.1
    255.255.255.0
4.
5.  ! Настройка портов 1-2
6.  interface range eth1/1-2
7.  switchport access vlan 10
```

Пример настройки VLAN на GO

```
1. import "github.com/vishvananda/netlink"
2. eth1, _ := netlink.LinkByName("eth1/1")
3. vlan := &netlink.Vlan{
4.     LinkAttrs: netlink.LinkAttrs{
5.         Name: "vlan.10",
6.         ParentIndex: eth1.Attrs().Index,
7.     },
8.     VlanId: 10,
9. }
10. netlink.LinkAdd(vlan)
11. addr, _ := netlink.ParseAddr("192.168.10.1/24")
12. netlink.AddrAdd(vlan, addr)
13. netlink.LinkSetUp(vlan)
```

```
1. ! VLAN 10 - подсеть 1
2. interface vlan10
3. ip address 192.168.10.1
   255.255.255.0
4.
5. ! Настройка портов 1-2
6. interface range eth1/1-2
7. switchport access vlan 10
```

Пример настройки VLAN на GO

```
1. import "github.com/vishvananda/netlink"
2. eth1, _ := netlink.LinkByName("eth1/1")
3. vlan := &netlink.Vlan{
4.     LinkAttrs: netlink.LinkAttrs{
5.         Name:      "vlan.10",
6.         ParentIndex: eth1.Attrs().Index,
7.     },
8.     VlanId: 10,
9. }
10. netlink.LinkAdd(vlan)
11. addr, _ := netlink.ParseAddr("192.168.10.1/24")
12. netlink.AddrAdd(vlan, addr)
13. netlink.LinkSetUp(vlan)
```

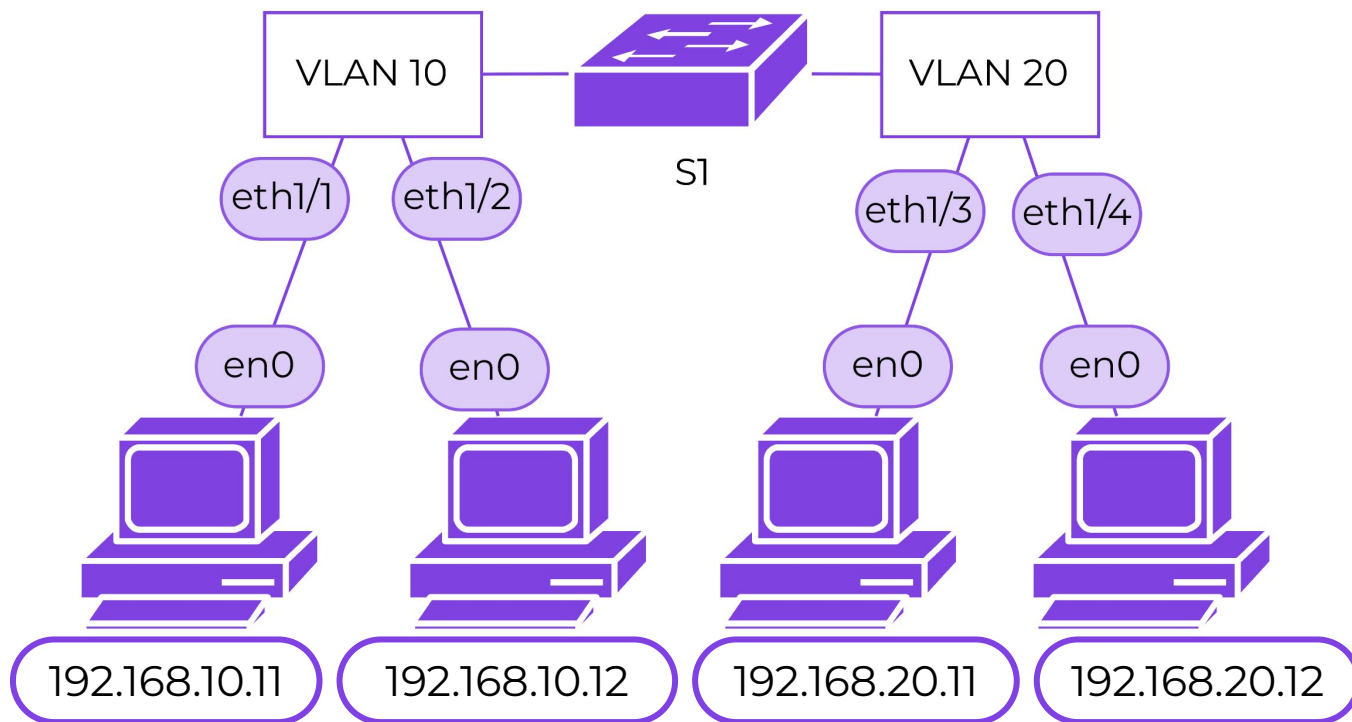
```
1. ! VLAN 10 - подсеть 1
2. interface vlan10
3. ip address 192.168.10.1
   255.255.255.0
4.
5. ! Настройка портов 1-2
6. interface range eth1/1-2
7. switchport access vlan 10
```

Пример настройки VLAN на GO

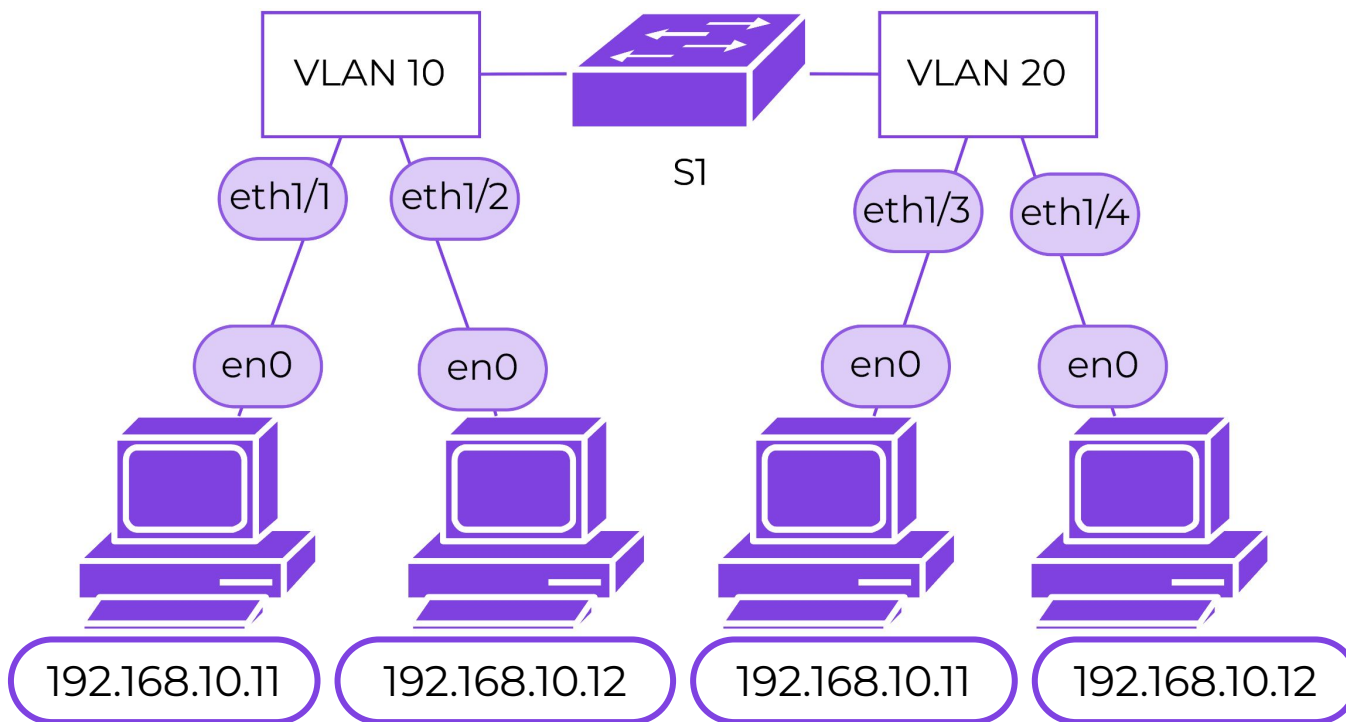
```
1. import "github.com/vishvananda/netlink"
2. eth1, _ := netlink.LinkByName("eth1/1")
3. vlan := &netlink.Vlan{
4.     LinkAttrs: netlink.LinkAttrs{
5.         Name:      "vlan.10",
6.         ParentIndex: eth1.Attrs().Index,
7.     },
8.     VlanId: 10,
9. }
10. netlink.LinkAdd(vlan)
11. addr, _ := netlink.ParseAddr("192.168.10.1/24")
12. netlink.AddrAdd(vlan, addr)
13. netlink.LinkSetUp(vlan)
```

```
1. ! VLAN 10 - подсеть 1
2. interface vlan10
3. ip address 192.168.10.1
   255.255.255.0
4.
5. ! Настройка портов 1-2
6. interface range eth1/1-2
7. switchport access vlan 10
```

Пример настройки VLAN



Пример настройки VLAN



Go netlink

Плюсы

- 1 Прimitives и модели для работы с устройством
- 2 Код на не любимом языке

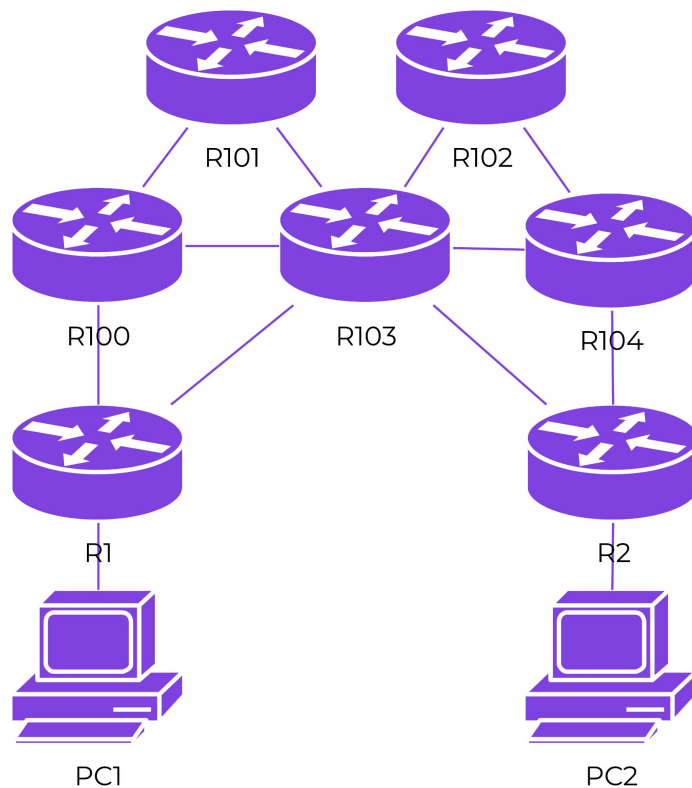
Минусы

- 1 Ну она работает только с Linux-сокетом
- 2 Если собираешь что-то необычное
- 3 Файлы всякие `/proc/sys/net/core/rmem_max`, в которых надо разобраться

**А теперь у нас
большая мегасеть**

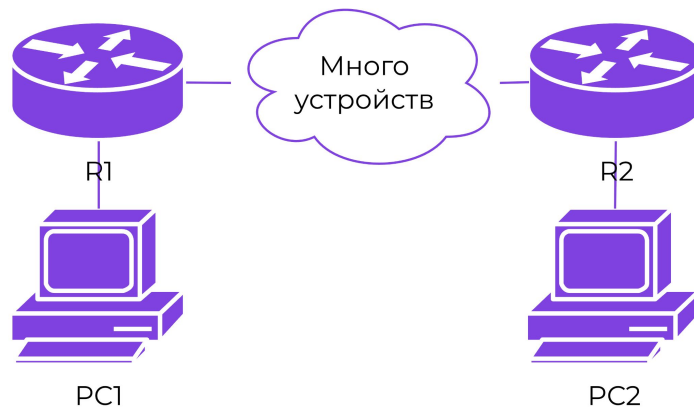
Мини-невменяемая сеть

точка банк

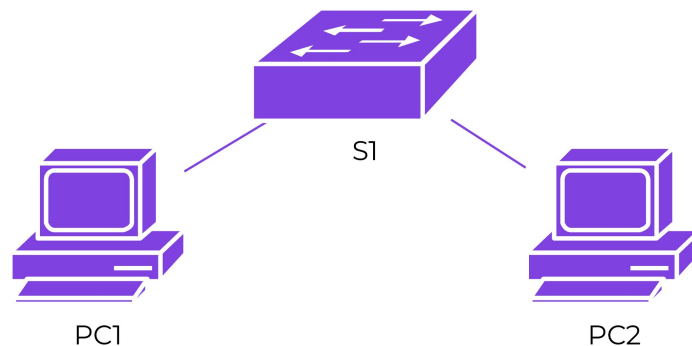


Теперь это просто мини-сеть

точка банк

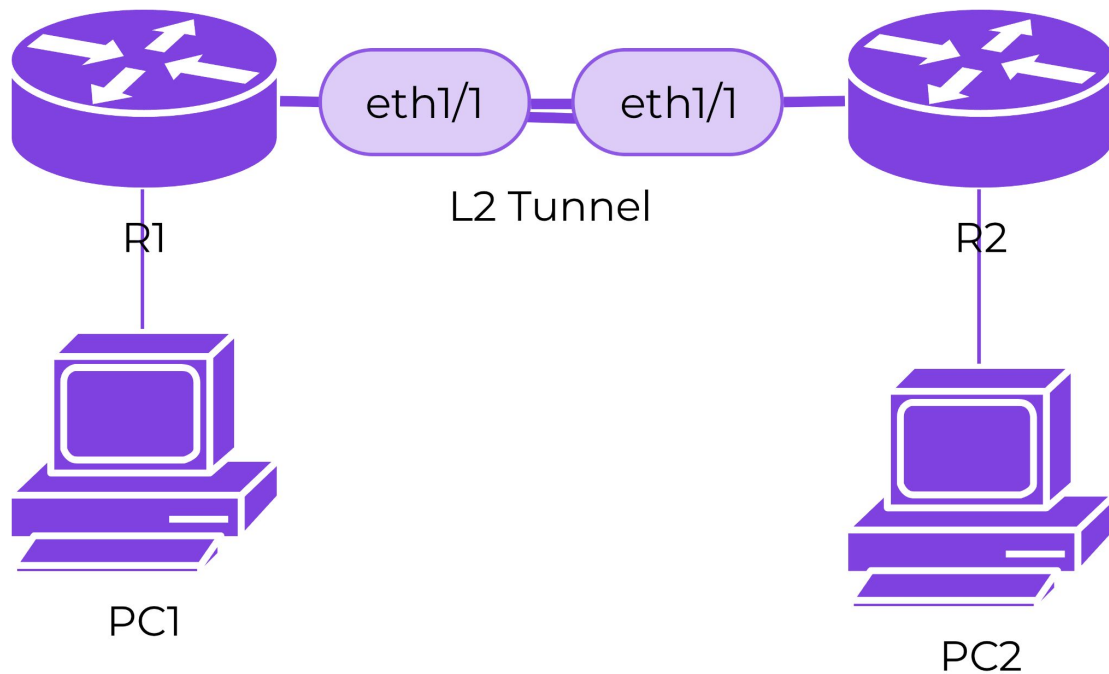


А мы хотим примерно это



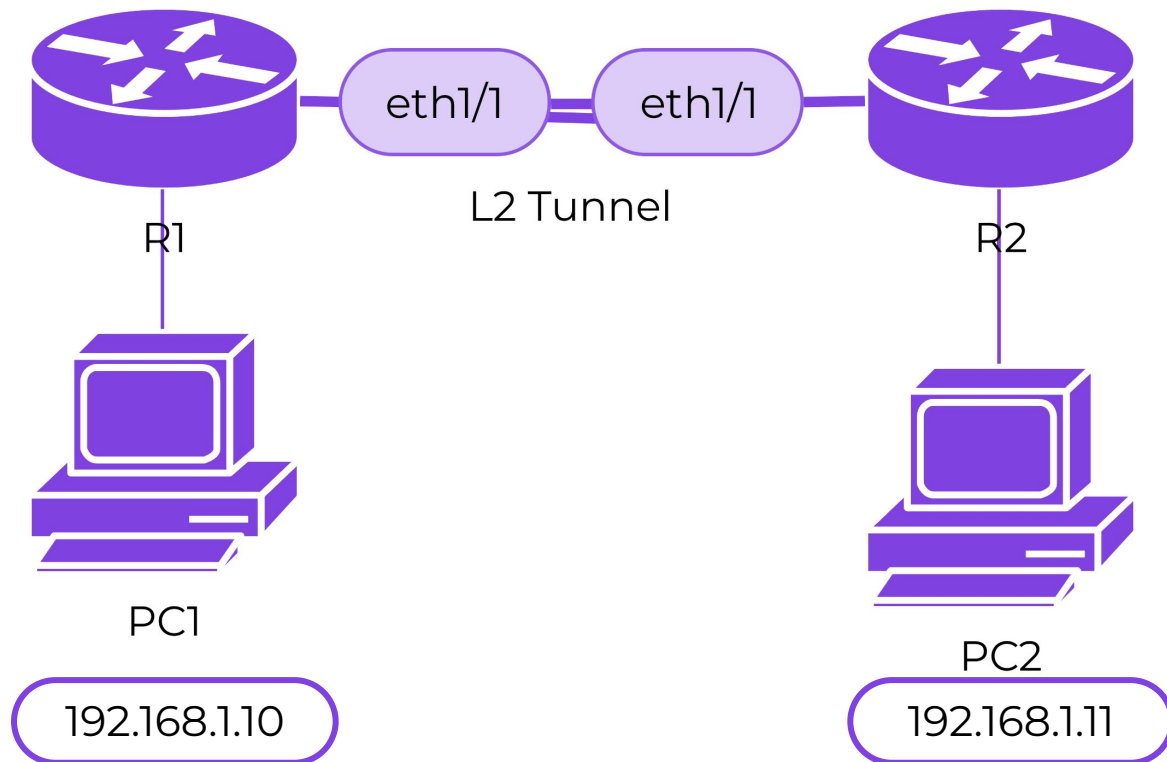
Туннелирование трафика

точка банк



Туннелирование трафика

точка банк



Туннелирование трафика L2TP

Плюсы

- 1 Совместимость на железе
- 2 Стандартные протоколы
- 3 4–8 байт на пакет заголовков → производительность

Минусы

- 1 Нет встроенного шифрования
- 2 Ограничения VLAN, нельзя бесконечно создать подсети

Туннелирование трафика VXLAN

Плюсы

- 1 16 млн VNI vs 4096 VLAN
- 2 Работа поверх любой сети
- 3 Возможность шифрования
- 4 Балансировка трафика

Минусы

- 1 Очень много метаданных → производительность
- 2 Сложная отладка пакетов и сложность в создании

Направить трафик в VXLAN на Go



SLURPEETH

slurpeeth

Очень примитивная, но, надеюсь, достаточно хорошая утилита для туннелирования TCP. Подключитесь к интерфейсу (интерфейсам), затем отправьте трафик в пункт назначения (пункты назначения) — вот и всё. Slurpeeth не знает и не заботится о том, что он отправляет. Это значит, что если вы эффективно настроите широковещательный домен (несколько интерфейсов или пунктов назначения), он будет слепо пересылать трафик от источника «а» во все пункты назначения (пункты назначения)

Направить трафик в VXLAN на Go

```
1.  config := slurpeeth.Config{
2.    Segments: []slurpeeth.Segment{{
3.      Name:      "server-tunnel",
4.      ID:        1000,
5.      Interfaces: []string{"en0"},
6.      Destinations: []string{"192.168.1.200:5000"}, }, }, },
7.  sm, _ := slurpeeth.GetManager(config)
8.  exitErr := make(chan bool)
9.  exitDone := make(chan bool)
10. _ = sm.RunDaemon(exitErr, exitDone)
11. select {
12. case <-exitDone:
13.   fmt.Println("Server stopped normally")
14. case <-exitErr:
15.   fmt.Println("Server stopped with error")
16. }
```

Направить трафик в VXLAN на Go

```
1.  config := slurpeeth.Config{
2.    Segments: []slurpeeth.Segment{{
3.      Name:      "server-tunnel",
4.      ID:        1000,
5.      Interfaces: []string{"en0"},
6.      Destinations: []string{"192.168.1.200:5000"}, }, }, },
7.  sm, _ := slurpeeth.GetManager(config)
8.  exitErr := make(chan bool)
9.  exitDone := make(chan bool)
10. _ = sm.RunDaemon(exitErr, exitDone)
11. select {
12. case <-exitDone:
13.   fmt.Println("Server stopped normally")
14. case <-exitErr:
15.   fmt.Println("Server stopped with error")
16. }
```

Направить трафик в VXLAN на Go

```
1.  config := slurpeeth.Config{
2.    Segments: []slurpeeth.Segment{{
3.      Name:      "server-tunnel",
4.      ID:        1000,
5.      Interfaces: []string{"en0"},
6.      Destinations: []string{"192.168.1.200:5000"}, }, }, }, }
7.  sm, _ := slurpeeth.GetManager(config)
8.  exitErr := make(chan bool)
9.  exitDone := make(chan bool)
10. _ = sm.RunDaemon(exitErr, exitDone)
11. select {
12. case <-exitDone:
13.   fmt.Println("Server stopped normally")
14. case <-exitErr:
15.   fmt.Println("Server stopped with error")
16. }
```

А поднимать тоннели мы уже умеем

```
1. parent, _ := netlink.LinkByName("eth1/1")
2. vxlan := &netlink.Vxlan{
3.     LinkAttrs: netlink.LinkAttrs{
4.         Name: "vxlan100",
5.     },
6.     VxlanId:      100,
7.     VtepDevIndex: parent.Attrs().Index,
8.     Group:         net.ParseIP("192.168.1.100"),
9. }
10. netlink.LinkAdd(vxlan)
11. vxlanInterface, err := netlink.LinkByName("vxlan100")
12. addr, _ := netlink.ParseAddr("10.0.100.1/24")
13. err = netlink.AddrAdd(vxlanInterface, addr)
14. err = netlink.LinkSetUp(vxlanInterface)
```

А поднимать тоннели мы уже умеем

```
1. parent, _ := netlink.LinkByName("eth1/1")
2. vxlan := &netlink.Vxlan{
3.     LinkAttrs: netlink.LinkAttrs{
4.         Name: "vxlan100",
5.     },
6.     VxlanId: 100,
7.     VtepDevIndex: parent.Attrs().Index,
8.     Group: net.ParseIP("192.168.1.100"),
9. }
10. netlink.LinkAdd(vxlan)
11. vxlanInterface, err := netlink.LinkByName("vxlan100")
12. addr, _ := netlink.ParseAddr("10.0.100.1/24")
13. err = netlink.AddrAdd(vxlanInterface, addr)
14. err = netlink.LinkSetUp(vxlanInterface)
```


А поднимать тоннели мы уже умеем

```
1. parent, _ := netlink.LinkByName("eth1/1")
2. vxlan := &netlink.Vxlan{
3.     LinkAttrs: netlink.LinkAttrs{
4.         Name: "vxlan100",
5.     },
6.     VxlanId:      100,
7.     VtepDevIndex: parent.Attrs().Index,
8.     Group:         net.ParseIP("192.168.1.100"),
9. }
10. netlink.LinkAdd(vxlan)
11. vxlanInterface, err := netlink.LinkByName("vxlan100")
12. addr, _ := netlink.ParseAddr("10.0.100.1/24")
13. err = netlink.AddrAdd(vxlanInterface, addr)
14. err = netlink.LinkSetUp(vxlanInterface)
```

А поднимать тоннели мы уже умеем

```
1. parent, _ := netlink.LinkByName("eth1/1")
2. vxlan := &netlink.Vxlan{
3.     LinkAttrs: netlink.LinkAttrs{
4.         Name: "vxlan100",
5.     },
6.     VxlanId:      100,
7.     VtepDevIndex: parent.Attrs().Index,
8.     Group:        net.ParseIP("192.168.1.100"),
9. }
10. netlink.LinkAdd(vxlan)
11. vxlanInterface, err := netlink.LinkByName("vxlan100")
12. addr, _ := netlink.ParseAddr("10.0.100.1/24")
13. err = netlink.AddrAdd(vxlanInterface, addr)
14. err = netlink.LinkSetUp(vxlanInterface)
```

Поднимаем тоннели в оборудовании

Поднимаем тоннели в K8S



go report A+ License MIT

Source Code: <https://github.com/scrapli/scrapligo>

Examples: <https://github.com/scrapli/scrapligo/tree/main/examples>

Go Docs: <https://pkg.go.dev/github.com/scrapli/scrapligo>

scrapligo -- scrap(e)li (but in go!) -- is a Go library focused on connecting to devices, specifically network devices (routers/switches/firewalls/etc.) via SSH and NETCONF.

Поднимаем тоннели в оборудовании

```
1.  d, _ := network.NewDriver(  
2.      "sandbox-iosxe-latest-1.cisco.com",  
3.      options.WithAuthUsername("developer"),  
4.      options.WithAuthPassword("Cisco12345"),  
5.  )  
6.  _ = d.Open()  
7.  defer d.Close()  
8.  prompt, _ := d.Channel.GetPrompt()  
9.  fmt.Printf("Подключились к: %s\n", prompt)  
10. r, err := d.SendCommand("show version")  
11. if err != nil {  
12.     fmt.Printf("Ошибка команды: %v\n", err)  
13.     return  
14. }  
15. fmt.Printf("Результат:\n%s\n", r.Result)
```

Поднимаем тоннели в оборудовании

```
1.  d, _ := network.NewDriver(  
2.    "sandbox-iosxe-latest-1.cisco.com",  
3.    options.WithAuthUsername("developer"),  
4.    options.WithAuthPassword("Cisco12345"),  
5.  )  
6.  _ = d.Open()  
7.  defer d.Close()  
8.  prompt, _ := d.Channel.GetPrompt()  
9.  fmt.Printf("Подключились к: %s\n", prompt)  
10. r, err := d.SendCommand("show version")  
11. if err != nil {  
12.   fmt.Printf("Ошибка команды: %v\n", err)  
13.   return  
14. }  
15. fmt.Printf("Результат:\n%s\n", r.Result)
```

Поднимаем тоннели в оборудовании

```
1.  d, _ := network.NewDriver(  
2.    "sandbox-iosxe-latest-1.cisco.com",  
3.    options.WithAuthUsername("developer"),  
4.    options.WithAuthPassword("Cisco12345"),  
5.  )  
6.  _ = d.Open()  
7.  defer d.Close()  
8.  prompt, _ := d.Channel.GetPrompt()  
9.  fmt.Printf("Подключились к: %s\n", prompt)  
10. r, err := d.SendCommand("show version")  
11. if err != nil {  
12.   fmt.Printf("Ошибка команды: %v\n", err)  
13.   return  
14. }  
15. fmt.Printf("Результат:\n%s\n", r.Result)
```

Поднимаем тоннели в оборудовании

```
1.  d, _ := network.NewDriver(  
2.    "sandbox-iosxe-latest-1.cisco.com",  
3.    options.WithAuthUsername("developer"),  
4.    options.WithAuthPassword("Cisco12345"),  
5.  )  
6.  _ = d.Open()  
7.  defer d.Close()  
8.  prompt, _ := d.Channel.GetPrompt()  
9.  fmt.Printf("Подключились к: %s\n", prompt)  
10. r, err := d.SendCommand("show version")  
11. if err != nil {  
12.   fmt.Printf("Ошибка команды: %v\n", err)  
13.   return  
14. }  
15. fmt.Printf("Результат:\n%s\n", r.Result)
```


Выдыхаем

Обобщаем

Плюсы

- 1 Инструментов достаточно много
- 2 Go умеет и в SDK, и в сокет
- 3 Библиотеки работают и покрыты тестами

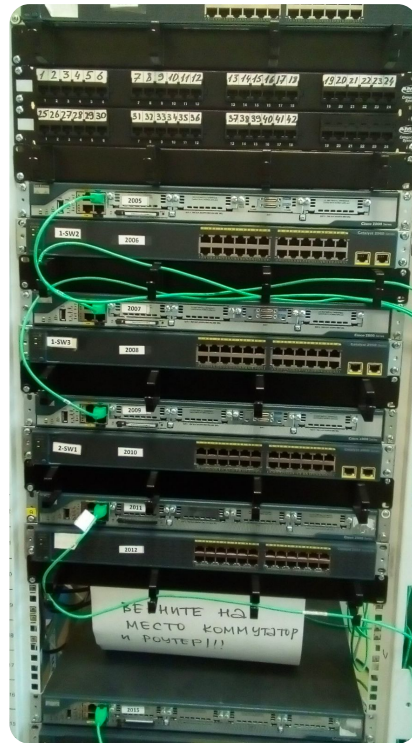
Минусы

- 1 Не очень популярные
- 2 Тесты на моках или системных файлах
- 3 SDK не от разработчиков оборудования

Собираем железный комплекс



The image shows a Juniper NetScreen-25 firewall unit with two M71 modules. The top module is labeled 'M71-1' and the bottom 'M71-2'. Both modules have labels indicating they are running Juniper M71 software. The unit is connected to a network switch via Ethernet cables.



Что нам нужно?

- 1 Создавать VLAN-подсети автоматически на физическом оборудовании
- 2 По запросу выдавать IP-адреса для подключения к оборудованию
- 3 Валидировать и проверять собранную топологию

Можем на Golang?

```
1.  variable "switches" {
2.    description = "Конфигурация коммутаторов"
3.    type = map(object({
4.      hostname = string
5.      interfaces = list(object({
6.        name      = string
7.        description = string
8.        mode       = string
9.        vlan       = number
10.     cdp_enable  = bool
11.     }))
12.  })}
```

Можем на Golang?

```
1.  variable "switches" {
2.    description = "Конфигурация коммутаторов"
3.    type = map(object({
4.      hostname = string
5.      interfaces = list(object({
6.        name      = string
7.        description = string
8.        mode       = string
9.        vlan       = number
10.     cdp_enable  = bool
11.   })))
12. })))
```

Можем на Terraform!

```
1.  hostname = "KK-344"
2.  interfaces = [
3.    {
4.      name          = "Fa1/0/36"
5.      description   = "Настройка порта Fa1/0/36 в VLAN 71"
6.      mode          = "access"
7.      vlan          = 71
8.      cdp_enable    = false
9.    },
10.   {
11.     name          = "Fa1/0/37"
12.     description   = "Настройка порта Fa1/0/37 в VLAN 30"
13.     mode          = "access"
14.     vlan          = 30
15.     cdp_enable    = false
```


Можем на Terraform!

```
1.  hostname = "KK-344"
2.  interfaces = [
3.    {
4.      name          = "Fa1/0/36"
5.      description   = "Настройка порта Fa1/0/36 в VLAN 71"
6.      mode          = "access"
7.      vlan          = 71
8.      cdp_enable    = false
9.    },
10.   {
11.     name          = "Fa1/0/37"
12.     description   = "Настройка порта Fa1/0/37 в VLAN 30"
13.     mode          = "access"
14.     vlan          = 30
15.     cdp_enable    = false
```

Можем на Go и Terraform!

```
1.  const configTemplate = `!  
2.  hostname {{.Hostname}}  
3.  !  
4.  {{range .Interfaces}}  
5.  ! {{.Description}}  
6.  interface {{.Name}}  
7.  switchport mode {{.Mode}}  
8.  switchport access vlan {{.VLAN}}  
9.  {{if not .CDPEnable}}no cdp enable{{else}}cdp enable{{end}}  
10. !  
11. {{end}}  
12. end  
13. `  
14.  
15. func main() {  
16.     tmpl := template.Must(template.New( "ios-config" ).Parse(configTemplate))  
17.     tmpl.Execute(os.Stdout, config)  
18. }
```

Можем на Go и Terraform!

```
1.  const configTemplate = `!  
2.  hostname {{.Hostname}}  
3.  !  
4.  {{range .Interfaces}}  
5.  ! {{.Description}}  
6.  interface {{.Name}}  
7.  switchport mode {{.Mode}}  
8.  switchport access vlan {{.VLAN}}  
9.  {{if not .CDPenable}}no cdp enable{{else}}cdp enable{{end}}  
10. !  
11. {{end}}  
12. end  
13. `  
14.  
15. func main() {  
16.     tpl := template.Must(template.New( "ios-config" ).Parse(configTemplate))  
17.     tpl.Execute(os.Stdout, config)  
18. }
```

Можем на Go и Terraform!

```
1.  const configTemplate = `!  
2.  hostname {{.Hostname}}  
3.  !  
4.  {{range .Interfaces}}  
5.  ! {{.Description}}  
6.  interface {{.Name}}  
7.  switchport mode {{.Mode}}  
8.  switchport access vlan {{.VLAN}}  
9.  {{if not .CDPEnable}}no cdp enable{{else}}cdp enable{{end}}  
10. !  
11. {{end}}  
12. end  
13. `  
14.  
15. func main() {  
16.     tmpl := template.Must(template.New( "ios-config" ).Parse(configTemplate))  
17.     tmpl.Execute(os.Stdout, config)  
18. }
```

Сетевая академия

Плюсы

- 1 Физическое железо
- 2 Его ещё и трогать нельзя
- 3 Работает в 99% случаев

Минусы

- 1 Работать можно только очно
- 2 Нужно понимать все нюансы работы сети
- 3 Нет GUI для студентов и автоматических проверок
- 4 Физическое оборудование ограничено и не разнообразно

**А может есть
готовый образ?**

Инструменты

Назв.	Тип	Доступность в РФ	Банк заданий	Тренд
GNS3	Эмулятор	Полная	Бесплатный	↓
PNETLab	Гибридная	Полная	Ограниченный	↓↑
Cisco CML	Гибридная	Ограниченная	Платный	↑
Packet Tracer	Симулятор	Ограниченная	Платный	-
NDG Netlab+	Физическая	Ограниченная	Платный	↓
Containerlab	Платформа	Полная	GitHub	↑

If you like this project, consider supporting it by buying me a coffee!

Donate   

NAME	LAST MODIFIED	SIZE	ACTIONS	☐		
 6Wind Turbo vRouter	2022-12-21 12:19:23	15.5 GB				
 A10 vThunder	2025-02-15 11:32:26	31.1 GB				
 a10-vThunder_410_P3	2022-12-21 12:19:07	5.68 GB				
 a10-vThunder_411	2022-12-21 12:19:15	9.35 GB				
 a10-vThunder_5_2_0-p1_38	2022-12-21 12:19:09	1.93 GB				
 acs-5.6	2022-12-21 12:29:44	2.68 GB				
 Alcatel&Nokia VSR 7550	2022-12-21 12:17:59	1.61 GB				
 Alienvault Cybersecurity OSSIM	2023-04-08 13:39:48	711 MB				
 alpine-3-16-2-base	2023-04-08 13:09:50	51.2 MB				

Утилита

github.com/ishare2-org/ishare2-cli

```
wget -qO- https://ishare2.sh/install | bash
ishare2 [action] [param1] [param2]
action:
  search      : Search for images by type
  pull        : Download an image by type and number
  installed   : Show installed images on server
  labs        : Show labs on server and download images for
  mylabs      : Same as labs command but using a customized
  relicense   : Generate a new iourc license for bin images
  upgrade     : Retrieves a menu that allows users to upgr
  changelog   : Show the latest changes made to ishare2
  gui         : Web app to use ishare2 in browser
  help        : Show useful information
  test        : Test if ishare2 dependencies are reachable
param1:
  type = all, bin, qemu, dynamips, docker or name
param2:
  number = This number can be obtained using ishare2 search
```

Поддержка разного типа образов:

- QEMU — полные виртуальные машины с ОС
- Docker — легковесные контейнеры приложений
- Bin/IOL — образы для сетевого эмулятора Cisco
- Dynamips — образы для старого эмулятора Cisco
- ISO — загрузочные образы для установки или Live-среды

Утилита

github.com/ishare2-org/ishare2-cli

```
wget -qO- https://ishare2.sh/install | bash
ishare2 [action] [param1] [param2]
action:
  search      : Search for images by type
  pull        : Download an image by type and number
  installed   : Show installed images on server
  labs        : Show labs on server and download images for
  mylabs      : Same as labs command but using a customized
  relicense   : Generate a new iourc license for bin images
  upgrade     : Retrieves a menu that allows users to upgr
  changelog   : Show the latest changes made to ishare2
  gui         : Web app to use ishare2 in browser
  help        : Show useful information
  test        : Test if ishare2 dependencies are reachable
param1:
  type = all, bin, qemu, dynamips, docker or name
param2:
  number = This number can be obtained using ishare2 search
```

Поддержка разного типа образов:

- **QEMU** — полные виртуальные машины с ОС
- Docker — легковесные контейнеры приложений
- Bin/IOL — образы для сетевого эмулятора Cisco
- Dynamips — образы для старого эмулятора Cisco
- ISO — загрузочные образы для установки или Live-среды

Инструменты

Сайт	Оборудование	Тип образа	Вес
arista.com	Коммутаторы	VMDK for cEOS, vEOS; OCI	600 МБ
checkpoint.com	CloudGuard Network Security	QCOW2	4–9 ГБ
mikrotik.com	RouterBOARD — аппаратная платформа	ISO	65 МБ
nokia.com/srlinux	Коммутаторы	OCI	4 ГБ

Инструменты

Сайт	Оборудование	Тип образа	Вес
arista.com	Коммутаторы	VMDK for cEOS, vEOS; OCI	600 МБ
checkpoint.com	CloudGuard Network Security	QCOW2	4–9 ГБ
mikrotik.com	RouterBOARD — аппаратная платформа	ISO	65 МБ
nokia.com/srlinux	Коммутаторы	OCI	4 ГБ

PNETLab



The screenshot displays the PNETLab interface. On the left, a 'Check' window shows the status of network components. It includes a table with the following data:

#	Статус	Время	Создана
1	Ошибка	00:00:01	22:32 13.12.2024

Below the table, there are two sections with status indicators:

- Ожидает** (Waiting): Настройка адресов устройств (Device address configuration). Интерфейсам должны быть назначены IP адреса (Interfaces should be assigned IP addresses).
- Ожидает** (Waiting): Проверка связности устройств в локальной сети (Device connectivity check in the local network). Все узлы в локальной сети должны пинговать друг друга (All nodes in the local network should ping each other).

On the right, a network diagram shows a topology with two routers (R1, R2), two switches (S1, S2), and two virtual PCs (vPC1, vPC2). The connections are as follows:

- R1 (e0/1) is connected to R2 (e0/1).
- R1 (e0/0) is connected to S1 (e0/1).
- R2 (e0/0) is connected to S2 (e0/1).
- S1 (e0/0) is connected to vPC1 (eth1).
- S2 (e0/0) is connected to vPC2 (eth1).

At the bottom left, there is a 'CHECK' button with a checkmark icon. At the bottom right, there is a 'Physical' button with a magnifying glass icon.

Стек:

- Apache guacamole
- Linux bridge для виртуальных сетей
- Docker для OCI образов
- NAT и DHCP на iptables

```
1.  user, err := userQueries.Get(userPod)
2.  if err != nil {
3.      return err
4.  }
5.  lab, _ := labSessionQuery.GetByAttemptId(attemptId)
6.  if lab.LabSessionID != 0 {
7.      lab.AddJoinedUser(userPod)
8.      labSessionQuery.DB.Save(&lab)
9.      user.LabSession = &lab.LabSessionID
10.     userQueries.DB.Save(&user)
11.     return nil
12. }
13. lab = models.LabSession{
14.     LabSessionLID:    attemptId,
15.     LabSessionPod:    userPod,
16.     LabSessionJoined: fmt.Sprintf("%d", userPod),
17.     LabSessionPath:    unlFilePathServer,
18. }
19. labSessionQuery.Save(&lab)
20. user.LabSession = &lab.LabSessionID
21. userQueries.Save(&user)
```

```
1.  user, err := userQueries.Get(userPod)
2.  if err != nil {
3.      return err
4.  }
5.  lab, _ := labSessionQuery.GetByAttemptId(attemptId)
6.  if lab.LabSessionID != 0 {
7.      lab.AddJoinedUser(userPod)
8.      labSessionQuery.DB.Save(&lab)
9.      user.LabSession = &lab.LabSessionID
10.     userQueries.DB.Save(&user)
11.     return nil
12. }
13. lab = models.LabSession{
14.     LabSessionLID:    attemptId,
15.     LabSessionPod:    userPod,
16.     LabSessionJoined: fmt.Sprintf("%d", userPod),
17.     LabSessionPath:    unlFilePathServer,
18. }
19. labSessionQuery.Save(&lab)
20. user.LabSession = &lab.LabSessionID
21. userQueries.Save(&user)
```



```
1.  user, err := userQueries.Get(userPod)
2.  if err != nil {
3.      return err
4.  }
5.  lab, _ := labSessionQuery.GetByAttemptId(attemptId)
6.  if lab.LabSessionID != 0 {
7.      lab.AddJoinedUser(userPod)
8.      labSessionQuery.DB.Save(&lab)
9.      user.LabSession = &lab.LabSessionID
10.     userQueries.DB.Save(&user)
11.     return nil
12. }
13. lab = models.LabSession{
14.     LabSessionLID:    attemptId,
15.     LabSessionPod:    userPod,
16.     LabSessionJoined: fmt.Sprintf("%d", userPod),
17.     LabSessionPath:   unlFilePathServer,
18. }
19. labSessionQuery.Save(&lab)
20. user.LabSession = &lab.LabSessionID
21. userQueries.Save(&user)
```

```
1. token, err := client.SSOToken(  
2.     "authorization_code",  
3.     dto.RedirectURI,  
4.     dto.Code,  
5. )  
6. if err != nil {  
7.     return nil, err  
8. }  
9. if dto.State != token.State {  
10.    return nil, errors.New("invalid state")  
11. }  
12. user, err := client.SSOUserInfo(token.AccessToken)  
13. if err != nil {  
14.    return nil, err  
15. }  
16. if user.Email == "" {  
17.    return nil, errors.New("invalid user email")  
18. }  
19. roleDB, err := userRoleQueries.GetOrCreateDefault()  
20. if err != nil {  
21.    return nil, err  
22. }  
23. userDB, err := userQueries.GetOrCreateByEmail(models.User{  
24.     Email: user.Email,  
25. })  
26. return userDB.Cookie
```

```
1. token, err := client.SSOToken(  
2.     "authorization_code",  
3.     dto.RedirectURI,  
4.     dto.Code,  
5. )  
6. if err != nil {  
7.     return nil, err  
8. }  
9. if dto.State != token.State {  
10.    return nil, errors.New("invalid state")  
11. }  
12. user, err := client.SSOUserInfo(token.AccessToken)  
13. if err != nil {  
14.    return nil, err  
15. }  
16. if user.Email == "" {  
17.    return nil, errors.New("invalid user email")  
18. }  
19. roleDB, err := userRoleQueries.GetOrCreateDefault()  
20. if err != nil {  
21.    return nil, err  
22. }  
23. userDB, err := userQueries.GetOrCreateByEmail(models.User{  
24.     Email: user.Email,  
25. })  
26. return userDB.Cookie
```

```
1. token, err := client.SSOToken(  
2.     "authorization_code",  
3.     dto.RedirectURI,  
4.     dto.Code,  
5. )  
6. if err != nil {  
7.     return nil, err  
8. }  
9. if dto.State != token.State {  
10.     return nil, errors.New("invalid state")  
11. }  
12. user, err := client.SSOUserInfo(token.AccessToken)  
13. if err != nil {  
14.     return nil, err  
15. }  
16. if user.Email == "" {  
17.     return nil, errors.New("invalid user email")  
18. }  
19. roleDB, err := userRoleQueries.GetOrCreateDefault()  
20. if err != nil {  
21.     return nil, err  
22. }  
23. userDB, err := userQueries.GetOrCreateByEmail(models.User{  
24.     Email: user.Email,  
25. })  
26. return userDB.Cookie
```

```
1. token, err := client.SSOToken(  
2.     "authorization_code",  
3.     dto.RedirectURI,  
4.     dto.Code,  
5. )  
6. if err != nil {  
7.     return nil, err  
8. }  
9. if dto.State != token.State {  
10.    return nil, errors.New("invalid state")  
11. }  
12. user, err := client.SSOUserInfo(token.AccessToken)  
13. if err != nil {  
14.    return nil, err  
15. }  
16. if user.Email == "" {  
17.    return nil, errors.New("invalid user email")  
18. }  
19. roleDB, err := userRoleQueries.GetOrCreateDefault()  
20. if err != nil {  
21.    return nil, err  
22. }  
23. userDB, err := userQueries.GetOrCreateByEmail(models.User{  
24.     Email: user.Email,  
25. })  
26. return userDB.Cookie
```

Наши потребности

Что нам нужно?

- 1 Создавать подсети VLAN автоматически на физическом оборудовании
- 2 По запросу выдавать IP-адреса для подключения к оборудованию
- 3 Валидировать и проверять собранную топологию
- 4 Проверять доступ к порту у пользователя (роли и доступ)

Что нам нужно?

- 1 Создавать подсети VLAN автоматически на физическом оборудовании
- 2 По запросу выдавать IP-адреса для подключения к оборудованию
- 3 Валидировать и проверять собранную топологию
- 4 Проверять доступ к порту у пользователя (роли и доступ)

Что нам нужно?

- 1 Создавать подсети VLAN автоматически на физическом оборудовании
- 2 По запросу выдавать IP-адреса для подключения к оборудованию
- 3 Валидировать и проверять собранную топологию
- 4 Проверять доступ к порту у пользователя (роли и доступ)
- 5 GUI

Что нам нужно?

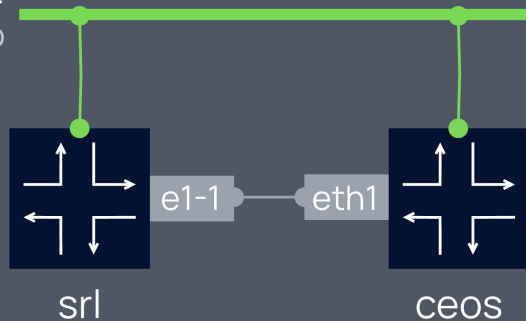
- 1 Создавать подсети VLAN автоматически на физическом оборудовании
- 2 По запросу выдавать IP-адреса для подключения к оборудованию
- 3 Валидировать и проверять собранную топологию
- 4 Проверять доступ к порту у пользователя (роли и доступ)
- 5 GUI
- 6 Открыть доступ во внешнюю сеть

Containerlab



```
curl -sL https://containerlab.dev/setup | sudo -E bash -s "all"
```

docker network:
clab



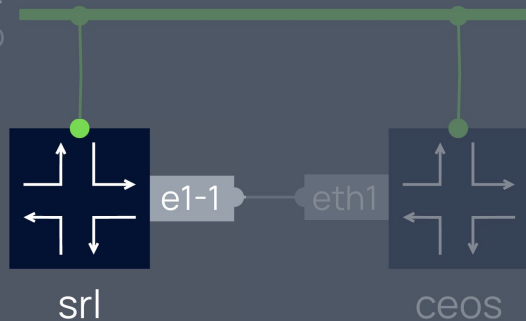
Lab logical diagram



topology-file.yaml

```
1 # topology documentation:
2 # http://containerlab.dev/lab-examples/srl-ceos/
3 name: srlceos01
4 topology:
5   nodes:
6     srl:
7       kind: nokia_srlinux
8       image: ghcr.io/nokia/srlinux:24.10
9     ceos:
10      kind: arista_ceos
11      image: ceos:4.32.0F
12   links:
13     - endpoints: [
14         "srl:ethernet-1/1",
15         "ceos:eth1"
16     ]
```

docker network:
clab



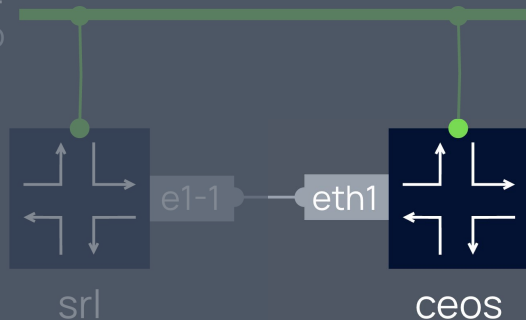
Lab logical diagram



topology-file.yaml

```
1 # topology documentation:
2 # http://containerlab.dev/lab-examples/srl-ceos/
3 name: srlceos01
4 topology:
5   nodes:
6     srl:
7       kind: nokia_srlinux
8       image: ghcr.io/nokia/srlinux:24.10
9     ceos:
10      kind: arista_ceos
11      image: ceos:4.32.0F
12   links:
13     - endpoints: [
14         "srl:ethernet-1/1",
15         "ceos:eth1"
16     ]
```

docker network:
clab

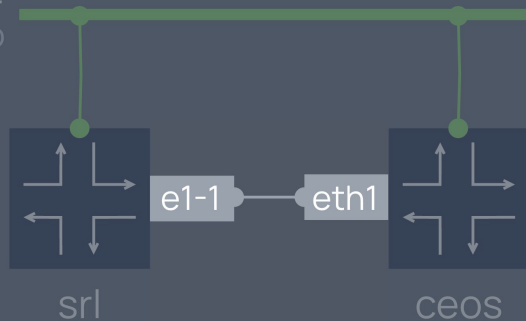


Lab logical diagram

topology-file.yaml

```
1 # topology documentation:
2 # http://containerlab.dev/lab-examples/srl-ceos/
3 name: srlceos01
4 topology:
5   nodes:
6     srl:
7       kind: nokia_srlinux
8       image: ghcr.io/nokia/srlinux:24.10
9     ceos:
10      kind: arista_ceos
11      image: ceos:4.32.0F
12   links:
13     - endpoints: [
14         "srl:ethernet-1/1",
15         "ceos:eth1"
16     ]
```

docker network:
clab



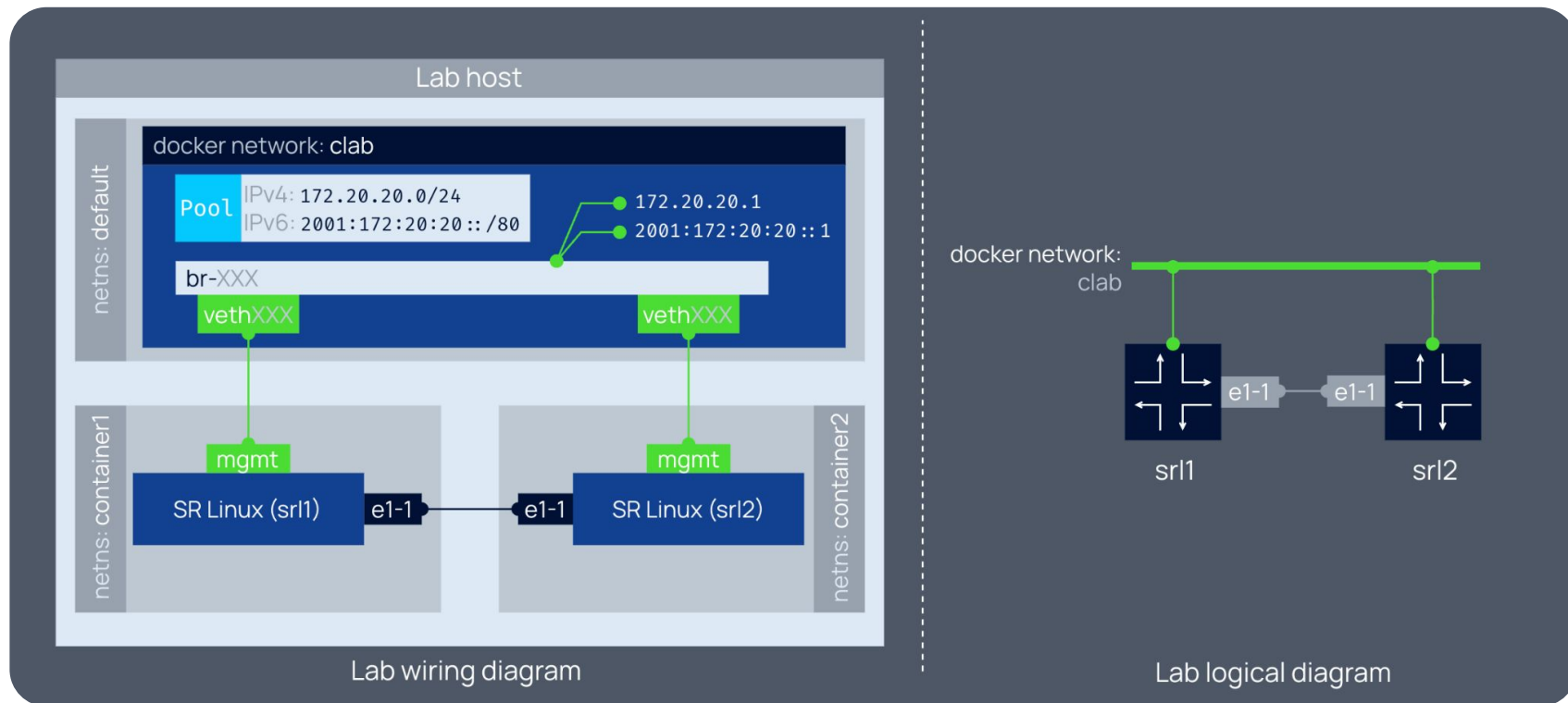
Lab logical diagram



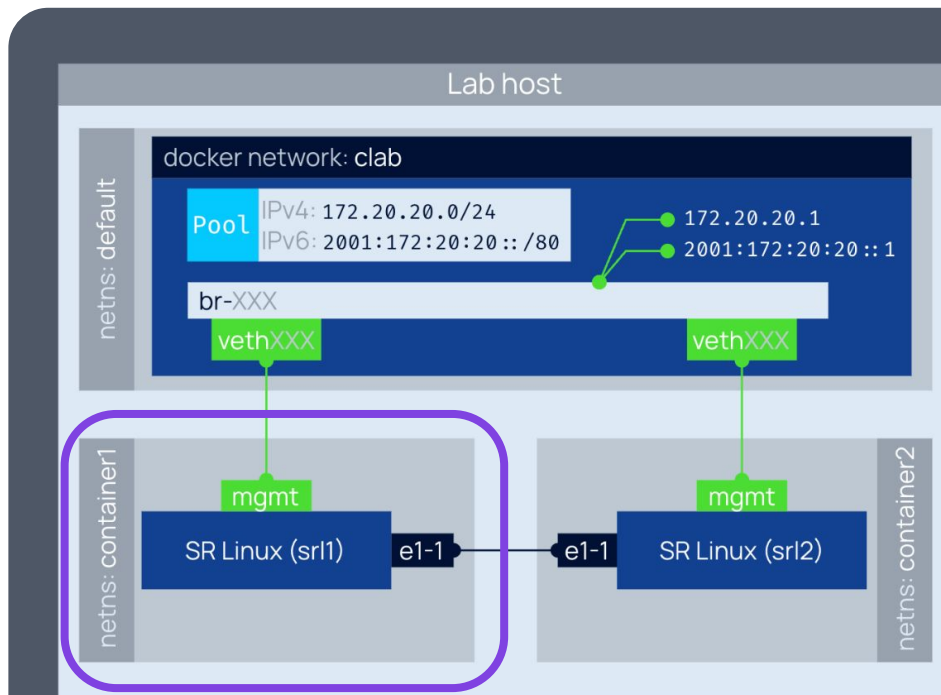
topology-file.yaml

```
1 # topology documentation:
2 # http://containerlab.dev/lab-examples/srl-ceos/
3 name: srlceos01
4 topology:
5   nodes:
6     srl:
7       kind: nokia_srlinux
8       image: ghcr.io/nokia/srlinux:24.10
9     ceos:
10      kind: arista_ceos
11      image: ceos:4.32.0F
12   links:
13     - endpoints: [
14       "srl:ethernet-1/1",
15       "ceos:eth1"
16     ]
```

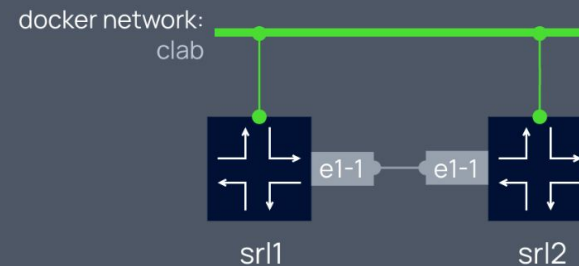

Что есть внутри?



Что есть внутри?

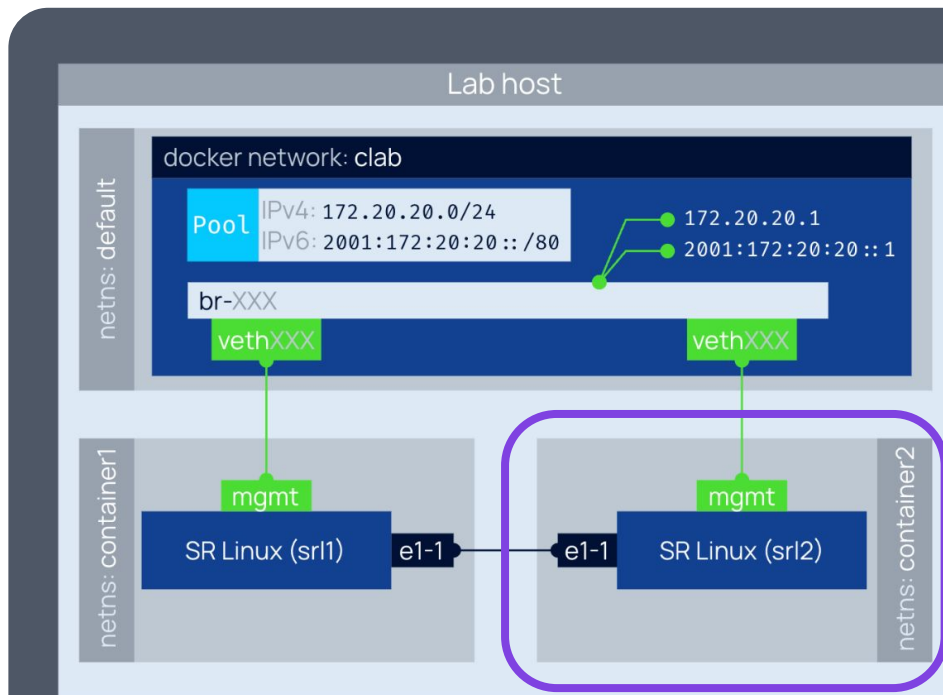


Lab wiring diagram

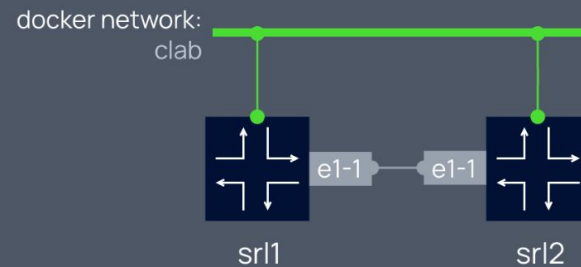


Lab logical diagram

Что есть внутри?

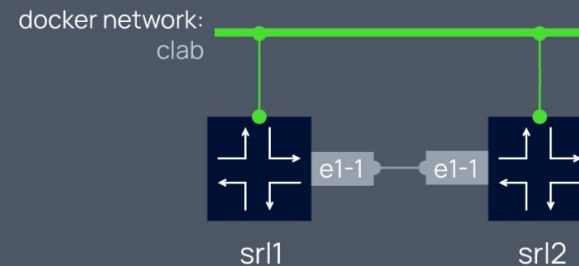
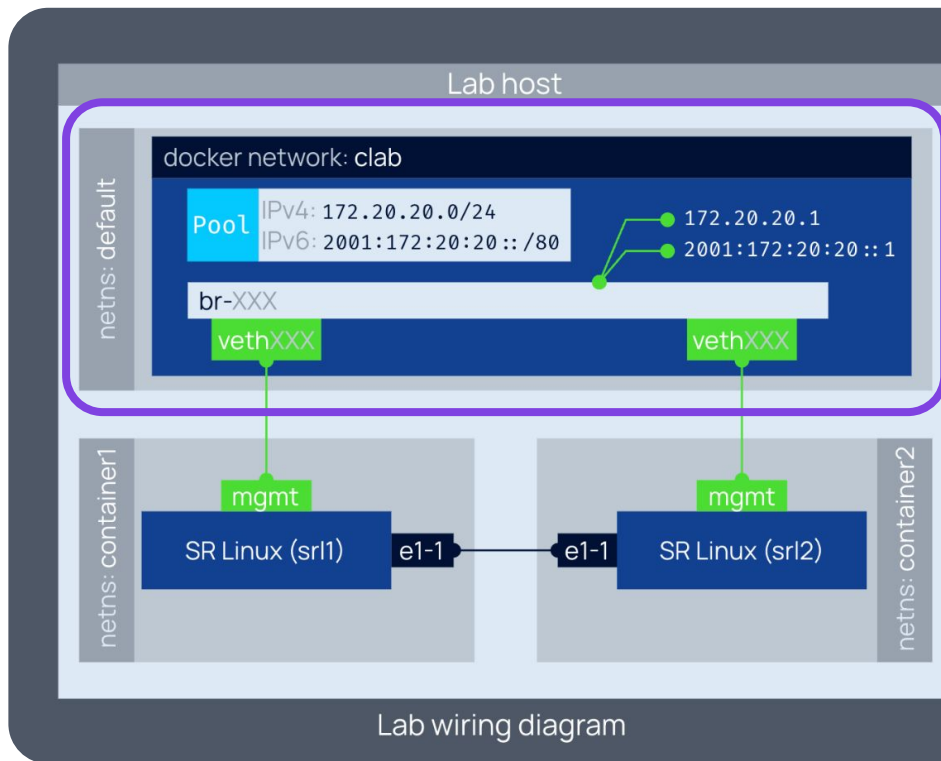


Lab wiring diagram



Lab logical diagram

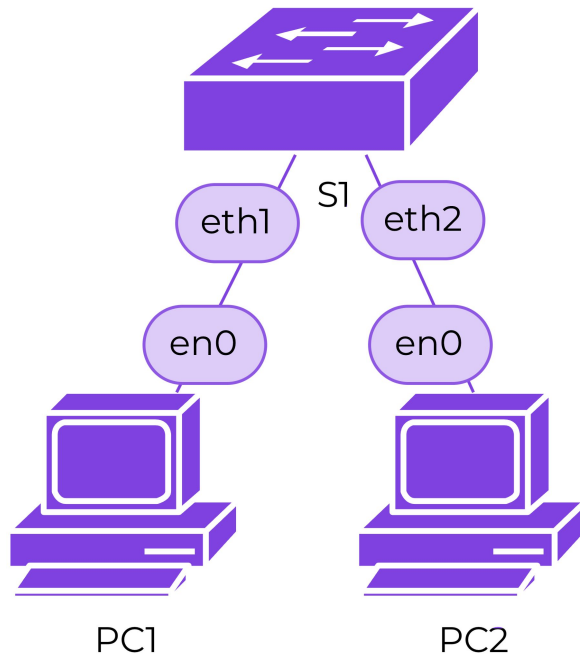
Что есть внутри?



На что похоже?

точка банк

IP	192.168.1.2
Маска	255.255.255.0
Шлюз	192.168.1.1



IP	192.168.1.3
Маска	255.255.255.0
Шлюз	192.168.1.1

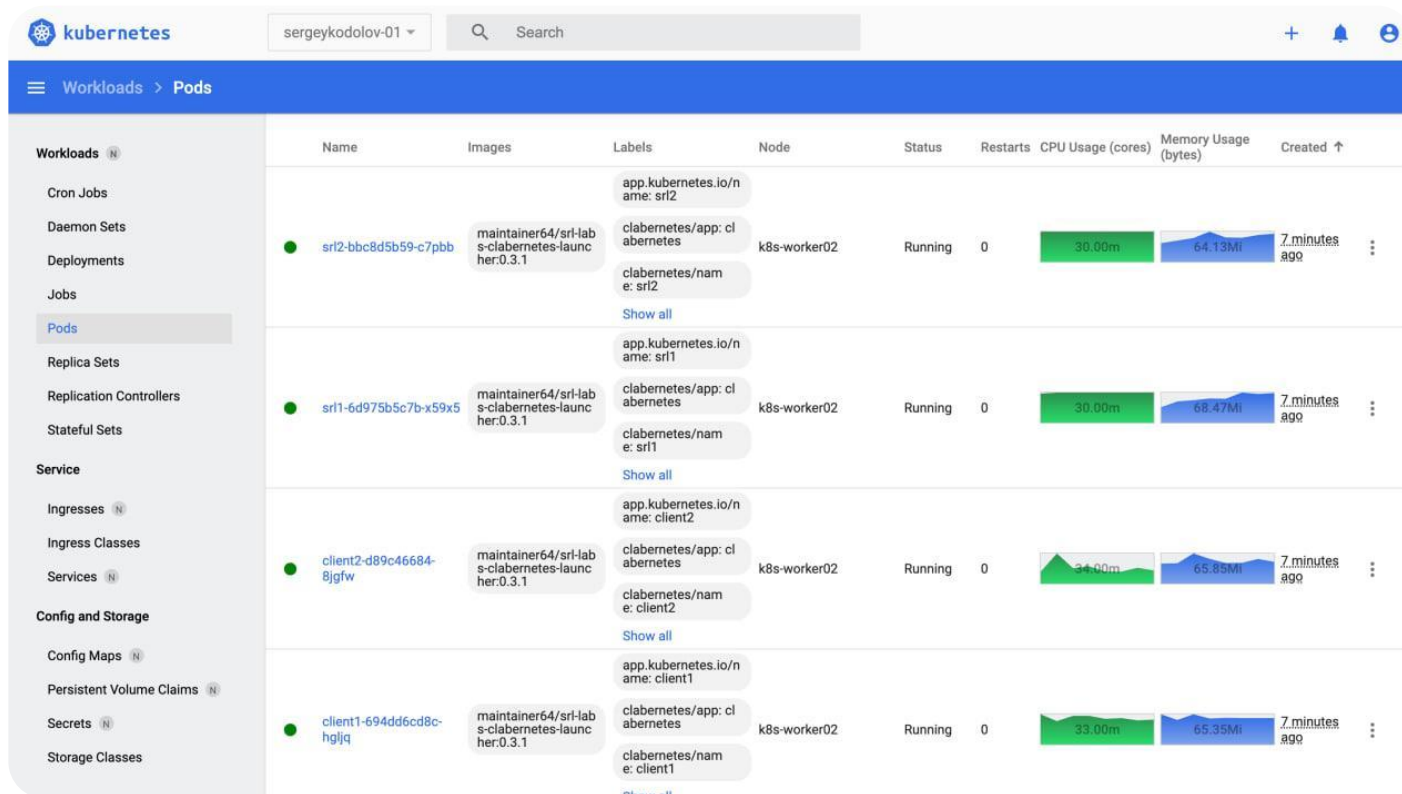
Поехали в Kubernetes

Containerlab.dev + Clabernetes

```
helm upgrade --install --namespace c9s \  
clabernetes oci://ghcr.io/srl-labs/clabernetes/clabernetes -f \  
c9s.values.yaml
```

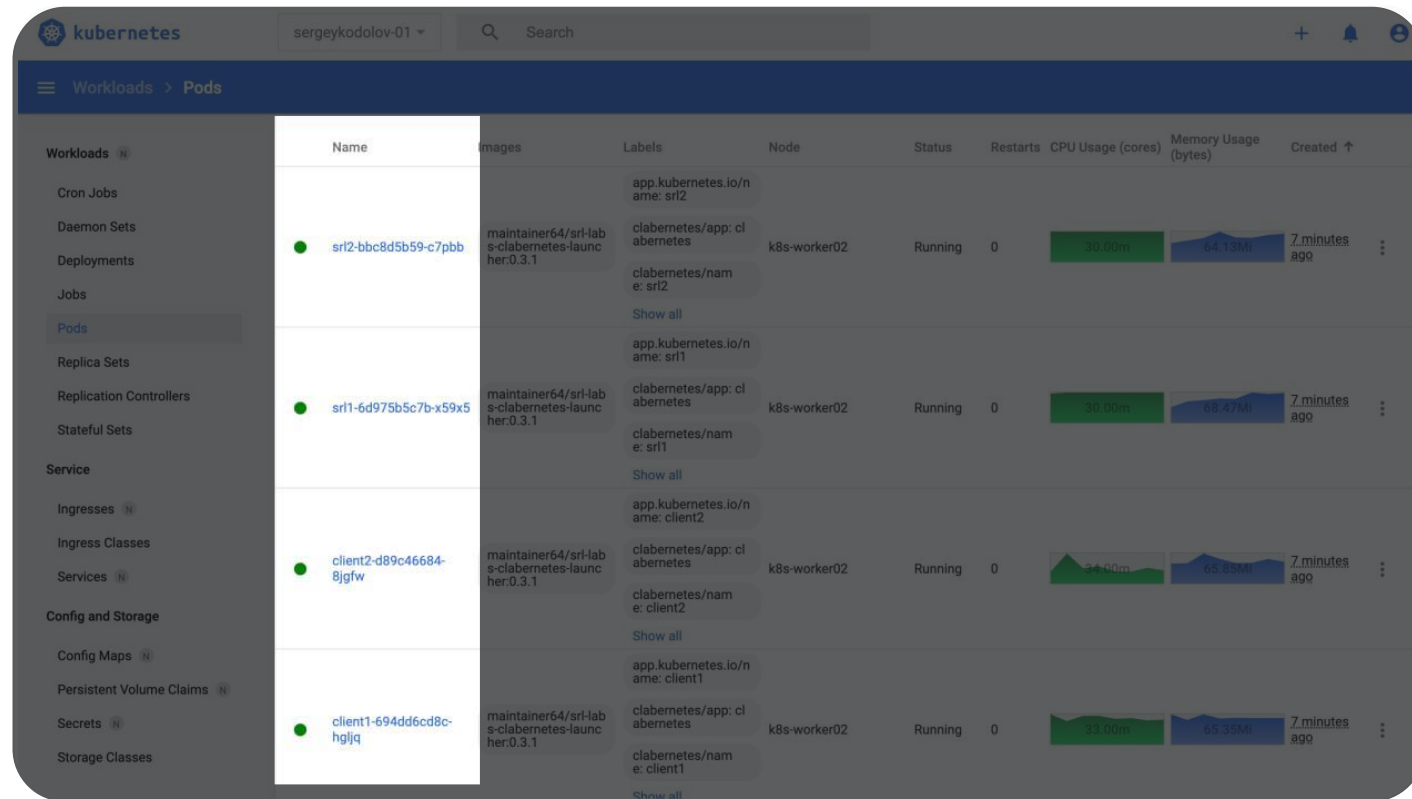
C9S в Kubernetes Dashboard

точка банк



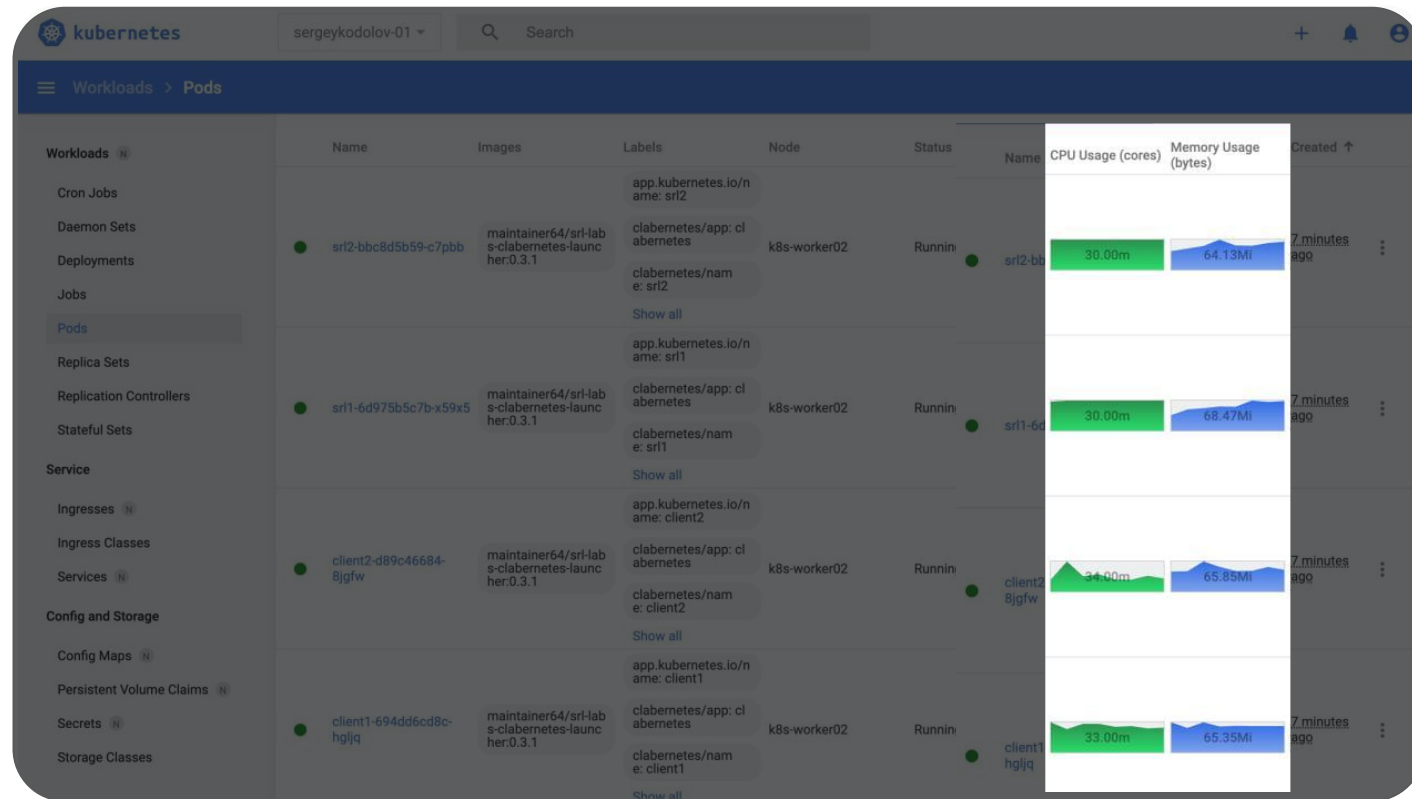
C9S в Kubernetes Dashboard

точка банк



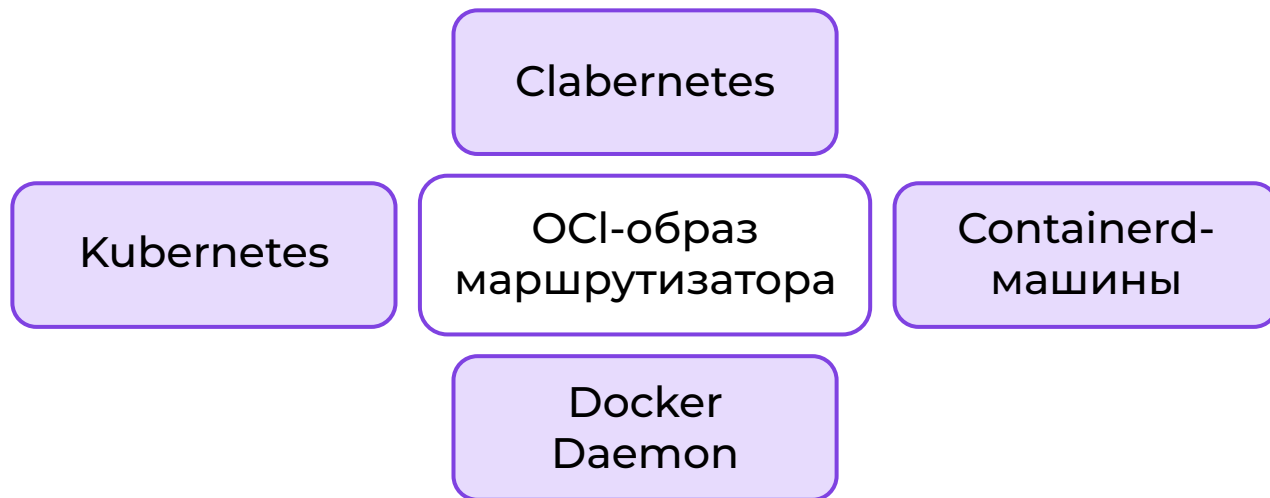
C9S в Kubernetes Dashboard

точка банк



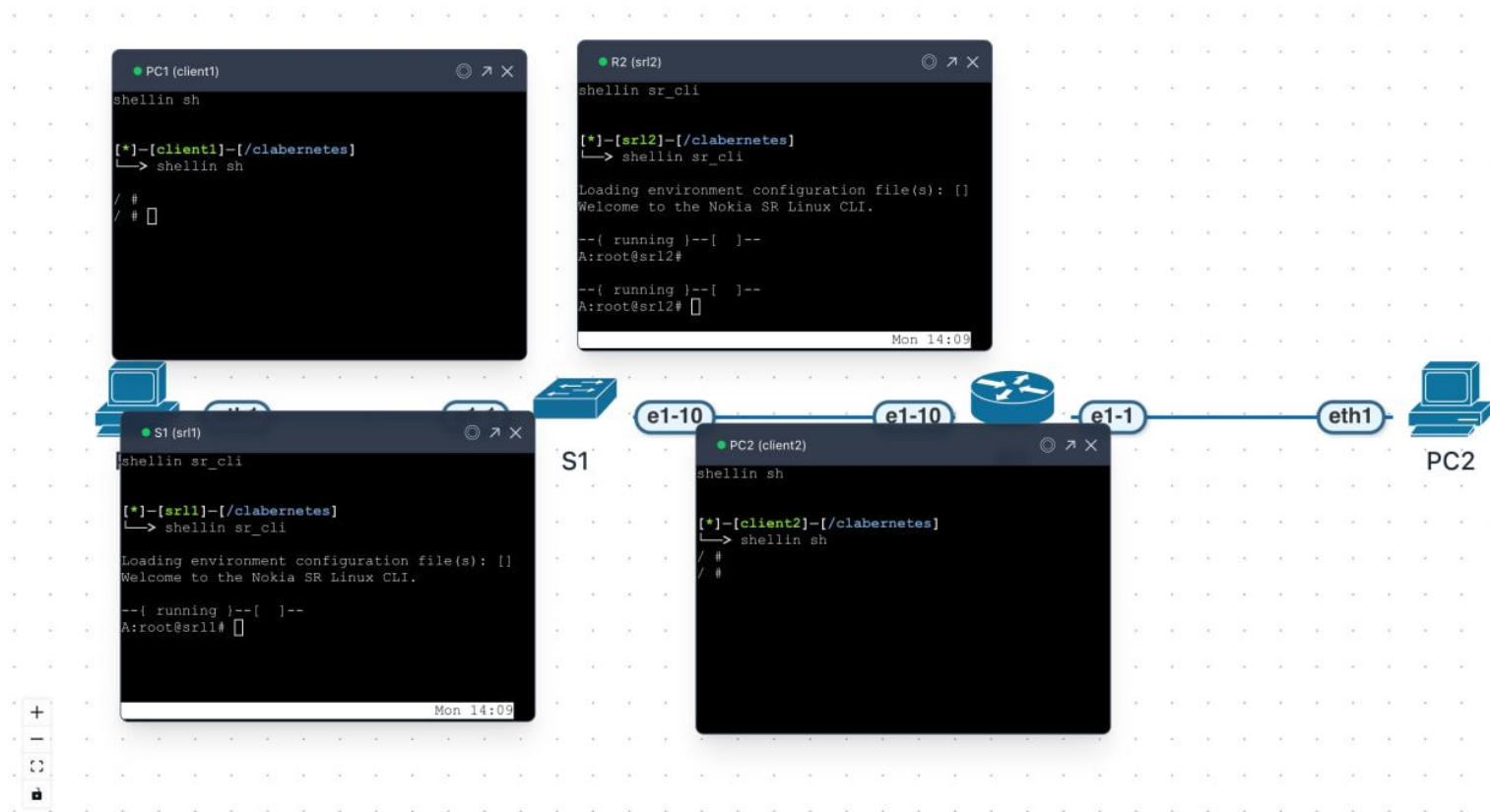
C9S внутри

точка банк



Консоль для работы

точка банк



```
1. // Render accepts the owning topology a mapping of clabernetes sub-topology configs
   and a node name
2. // and renders the final deployment for this node.
3. func (r *DeploymentReconciler) Render(
4.     owningTopology *clabernetesapisv1alpha1.Topology,
5.     clabernetesConfigs map[string]*clabernetesutilcontainerlab.Config,
6.     nodeName string,
7. ) *k8sapps1.Deployment {
8.     owningTopologyName := owningTopology.GetName()
9.     deploymentName := fmt.Sprintf("%s-%s", owningTopologyName, nodeName)
10.    if ResolveTopologyRemovePrefix(owningTopology) {
11.        deploymentName = nodeName
12.    }
13.    configVolumeName := fmt.Sprintf("%s-config", owningTopologyName)
14.    deployment := r.renderDeploymentBase(
15.        deploymentName,
16.        owningTopology.GetNamespace(),
17.        owningTopologyName,
18.        nodeName,
19.    )
20.
```

Код C9S

```
1. // Render accepts the owning topology a mapping of clabernetes sub-topology configs
   and a node name
2. // and renders the final deployment for this node.
3. func (r *DeploymentReconciler) Render(
4.     owningTopology *clabernetesapisv1alpha1.Topology,
5.     clabernetesConfigs map[string]*clabernetesutilcontainerlab.Config,
6.     nodeName string,
7. ) *k8sapps1.Deployment {
8.     owningTopologyName := owningTopology.GetName()
9.     deploymentName := fmt.Sprintf("%s-%s", owningTopologyName, nodeName)
10.    if ResolveTopologyRemovePrefix(owningTopology) {
11.        deploymentName = nodeName
12.    }
13.    configVolumeName := fmt.Sprintf("%s-config", owningTopologyName)
14.    deployment := r.renderDeploymentBase(
15.        deploymentName,
16.        owningTopology.GetNamespace(),
17.        owningTopologyName,
18.        nodeName,
19.    )
20.
```

Код C9S

точка банк

```
1.      r.renderDeploymentScheduling(
2.          deployment,
3.          owningTopology,
4.      )
5.
6.      volumeMountsFromCommonSpec := r.renderDeploymentVolumes(
7.          deployment,
8.          nodeName,
9.          configVolumeName,
10.         owningTopologyName,
11.         owningTopology,
12.     )
13.
14.     r.renderDeploymentContainer(
15.         deployment,
16.         nodeName,
17.         configVolumeName,
18.         volumeMountsFromCommonSpec,
19.         owningTopology,
20.     )
21.
22.     r.renderDeploymentContainerEnv(
23.         deployment,
24.         nodeName,
25.         owningTopologyName,
26.         owningTopology,
27.         ciabernetesConfigs,
28.     )
29.
30.     r.renderDeploymentContainerResources(
31.         deployment,
32.         nodeName,
33.         owningTopology,
34.         ciabernetesConfigs,
35.     )
36.
37.     r.renderDeploymentNodeSelectors(
38.         deployment,
39.         nodeName,
40.         owningTopology,
41.         ciabernetesConfigs,
42.     )
43.
44.     r.renderDeploymentContainerPrivileges(
45.         deployment,
46.         nodeName,
47.         owningTopology,
48.     )
49.
50.     r.renderDeploymentContainerStatus(
51.         deployment,
52.         nodeName,
53.         owningTopology,
54.     )
55.
56.     r.renderDeploymentDevices(
57.         deployment,
58.         owningTopology,
59.     )
60.
61.     r.renderDeploymentPersistence(
62.         deployment,
63.         nodeName,
64.         owningTopologyName,
65.         owningTopology,
66.     )
67.
68.     return deployment
69. }
```

```

1.  func (r *DeploymentReconciler) RenderAll(
2.      deployment,
3.      owningTopology,
4.  )
5.
6.      volumeMountsFromCommonSpec := r.renderDeploymentVolumes(
7.          deployment,
8.          nodeName,
9.          configVolumeName,
10.          owningTopologyName,
11.          owningTopology,
12.      )
13.
14.      r.renderDeploymentContainer(
15.          deployment,
16.          nodeName,
17.          configVolumeName,
18.          volumeMountsFromCommonSpec,
19.          owningTopology,
20.      )
21.
22.      r.renderDeploymentContainerEnv(
23.          deployment,
24.          nodeName,
25.          owningTopologyName,
26.          owningTopology,
27.          clabernetesConfigs,
28.      )
29.
30.      r.renderDeploymentContainerResources(
31.          deployment,
32.          nodeName,
33.          owningTopology,
34.          clabernetesConfigs,
35.      )
36.
37.      r.renderDeploymentNodeSelectors(
38.          deployment,
39.          nodeName,
40.          owningTopology,
41.          clabernetesConfigs,
42.      )
43.
44.      r.renderDeploymentContainerPrivileges(
45.          deployment,
46.          nodeName,
47.          owningTopology,
48.      )
49.
50.      r.renderDeploymentContainerStatus(
51.          deployment,
52.          nodeName,
53.          owningTopology,
54.      )
55.
56.      r.renderDeploymentDevices(
57.          deployment,
58.          owningTopology,
59.      )
60.
61.      r.renderDeploymentPersistence(
62.          deployment,
63.          nodeName,
64.          owningTopologyName,
65.          owningTopology,
66.      )
67.
68.      return deployment
69.  )

```

```

1.  func (r *DeploymentReconciler) RenderAll(
2.      owningTopology *clabernetesapisv1alpha1.Topology,
3.      clabernetesConfigs
4.      map[string]*clabernetesutilcontainerlab.Config,
5.      nodeNames []string,
6.  ) []*k8sapps1.Deployment {
7.      deployments := make([]*k8sapps1.Deployment,
8.          len(nodeNames))
9.
10.         for idx, nodeName := range nodeNames {
11.             deployments[idx] = r.Render(
12.                 owningTopology,
13.                 clabernetesConfigs,
14.                 nodeName,
15.             )
16.         }
17.         return deployments

```


Код Containerlab

```
1. func (c *clabernetes) startup() {  
2.   c.logger.Info("starting clabernetes...")  
3.   c.containerlabVersion()  
4.   c.setup()  
5.   c.image()  
6.   c.launch()  
7.   c.connectivity()
```

Код Containerlab

```
1. package cmd
2. import (
3.     "fmt"
4.     "net"
5.     "github.com/charmbracelet/log"
6.     "github.com/spf13/cobra"
7.     clablinks "github.com/srl-labs/containerlab/links"
8.     clabutils "github.com/srl-labs/containerlab/utils"
9.     "github.com/vishvananda/netlink"
10. )
11. func vxlanCmd(o *Options) (*cobra.Command, error) { //nolint: funlen
12.     c := &cobra.Command{
13.         Use:     "vxlan",
14.         Short:   "VxLAN interface commands",
15.     }
16.     vxlanCreateCmd := &cobra.Command{
17.         Use:     "create",
18.         Short:   "create vxlan interface",
19.         PreRunE: func(_ *cobra.Command, _ []string) error {
20.             return clabutils.CheckAndGetRootPrivs()
21.         },
```

Код Containerlab

```
1. package cmd
2. import (
3.     "fmt"
4.     "net"
5.     "github.com/charmbracelet/log"
6.     "github.com/spf13/cobra"
7.     clablinks "github.com/srl-labs/containerlab/links"
8.     clabutils "github.com/srl-labs/containerlab/utils"
9.     "github.com/vishvananda/netlink"
10. )
11. func vxlanCmd(o *Options) (*cobra.Command, error) { //nolint: funlen
12.     c := &cobra.Command{
13.         Use:     "vxlan",
14.         Short:   "VxLAN interface commands",
15.     }
16.     vxlanCreateCmd := &cobra.Command{
17.         Use:     "create",
18.         Short:   "create vxlan interface",
19.         PreRunE: func(_ *cobra.Command, _ []string) error {
20.             return clabutils.CheckAndGetRootPrivs()
21.         },
```

Код Containerlab

```
1.  ctx := cobraCmd.Context()
2.  if _, err := netlink.LinkByName(o.ToolsVxlan.Link); err != nil {
3.      return fmt.Errorf("failed to lookup link %q: %v",
4.          o.ToolsVxlan.Link, err)
5.  }
6.  // if vxlan device was not set specifically, we will use
7.  // the device that is reported by `ip route get $remote`
8.  if o.ToolsVxlan.ParentDevice == "" {
9.      r, err := clabutils.GetRouteForIP(net.ParseIP(o.ToolsVxlan.Remote))
10.     if err != nil {
11.         return fmt.Errorf(
12.             "failed to find a route to VxLAN remote address %s", o.ToolsVxlan.Remote,
13.         )
14.     }
15.     o.ToolsVxlan.ParentDevice = r.Interface.Name
16. }
17. vxlraw := &clablinks.LinkVxlanRaw{
18.     Remote:      o.ToolsVxlan.Remote,
19.     VNI:         int(o.ToolsVxlan.ID),
20.     ParentInterface: o.ToolsVxlan.ParentDevice,
21.     LinkCommonParams: clablinks.LinkCommonParams{
22.         MTU: int(o.ToolsVxlan.MTU),
23.     },
24.     DstPort: int(o.ToolsVxlan.DstPort),
25.     SrcPort: int(o.ToolsVxlan.SrcPort),
26.     LinkType: clablinks.LinkTypeVxlanStitch,
27.     ...
```

Код Containerlab

```
1. ctx := cobraCmd.Context()
2. if _, err := netlink.LinkByName(o.ToolsVxlan.Link); err != nil {
3.     return fmt.Errorf("failed to lookup link %q: %v",
4.         o.ToolsVxlan.Link, err)
5. }
6. // if vxlan device was not set specifically, we will use
7. // the device that is reported by `ip route get $remote`
8. if o.ToolsVxlan.ParentDevice == "" {
9.     r, err := clabutils.GetRouteForIP(net.ParseIP(o.ToolsVxlan.Remote))
10.    if err != nil {
11.        return fmt.Errorf(
12.            "failed to find a route to VxLAN remote address %s", o.ToolsVxlan.Remote,
13.        )
14.    }
15.    o.ToolsVxlan.ParentDevice = r.Interface.Name
16. }
17. vxlaw := &clablinks.LinkVxlanRaw{
18.    Remote:      o.ToolsVxlan.Remote,
19.    VNI:         int(o.ToolsVxlan.ID),
20.    ParentInterface: o.ToolsVxlan.ParentDevice,
21.    LinkCommonParams: clablinks.LinkCommonParams{
22.        MTU: int(o.ToolsVxlan.MTU),
23.    },
24.    DstPort: int(o.ToolsVxlan.DstPort),
25.    SrcPort: int(o.ToolsVxlan.SrcPort),
26.    LinkType: clablinks.LinkTypeVxlanStitch,
27.    ...
```

Код Containerlab

```
1.  ctx := cobraCmd.Context()
2.  if _, err := netlink.LinkByName(o.ToolsVxlan.Link); err != nil {
3.      return fmt.Errorf("failed to lookup link %q: %v",
4.          o.ToolsVxlan.Link, err)
5.  }
6.  // if vxlan device was not set specifically, we will use
7.  // the device that is reported by `ip route get $remote`
8.  if o.ToolsVxlan.ParentDevice == "" {
9.      r, err := clabutils.GetRouteForIP(net.ParseIP(o.ToolsVxlan.Remote))
10.     if err != nil {
11.         return fmt.Errorf(
12.             "failed to find a route to VxLAN remote address %s", o.ToolsVxlan.Remote,
13.         )
14.     }
15.     o.ToolsVxlan.ParentDevice = r.Interface.Name
16. }
17. vxlraw := &clablinks.LinkVxlanRaw{
18.     Remote:      o.ToolsVxlan.Remote,
19.     VNI:         int(o.ToolsVxlan.ID),
20.     ParentInterface: o.ToolsVxlan.ParentDevice,
21.     LinkCommonParams: clablinks.LinkCommonParams{
22.         MTU: int(o.ToolsVxlan.MTU),
23.     },
24.     DstPort: int(o.ToolsVxlan.DstPort),
25.     SrcPort: int(o.ToolsVxlan.SrcPort),
26.     LinkType: clablinks.LinkTypeVxlanStitch,
27.     ...
```

Код Containerlab

```
1.  rp := &clablinks.ResolveParams{
2.      Nodes: map[string]clablinks.Node{
3.          "host": clablinks.GetHostLinkNode(),
4.      },
5.      VxlanIfaceNameOverwrite: o.ToolsVxlan.Link,
6.  }
7.  link, err := vxlaw.Resolve(rp)
8.  if err != nil {
9.      return err
10. }
11. vxl, ok := link.(*clablinks.VxlanStitched)
12. if !ok {
13.     return fmt.Errorf("not a VxlanStitched link")
14. }
15. err = vxl.DeployWithExistingVeth(ctx)
16. if err != nil {
17.     return err
18. }
```

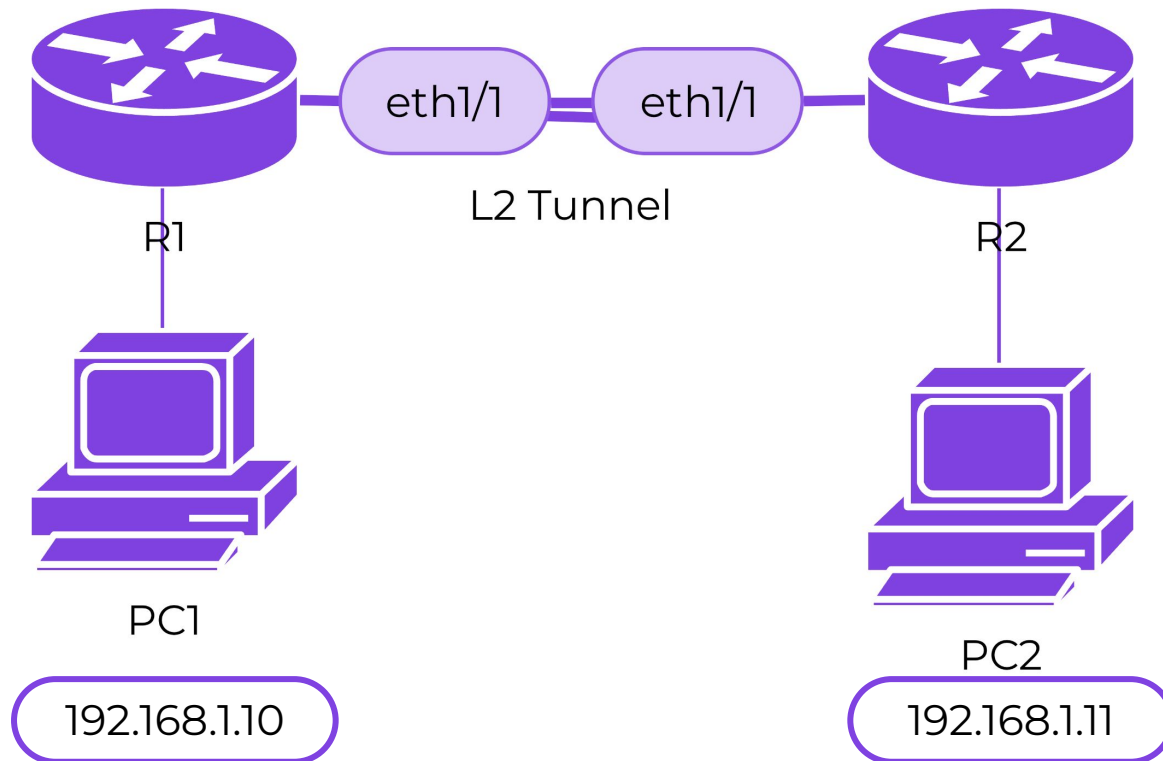
Код Containerlab

```
1.  rp := &clablinks.ResolveParams{
2.      Nodes: map[string]clablinks.Node{
3.          "host": clablinks.GetHostLinkNode(),
4.      },
5.      VxlanIfaceNameOverwrite: o.ToolsVxlan.Link,
6.  }
7.  link, err := vxlaw.Resolve(rp)
8.  if err != nil {
9.      return err
10. }
11. vxl, ok := link.(*clablinks.VxlanStitched)
12. if !ok {
13.     return fmt.Errorf("not a VxlanStitched link")
14. }
15. err = vxl.DeployWithExistingVeth(ctx)
16. if err != nil {
17.     return err
18. }
```

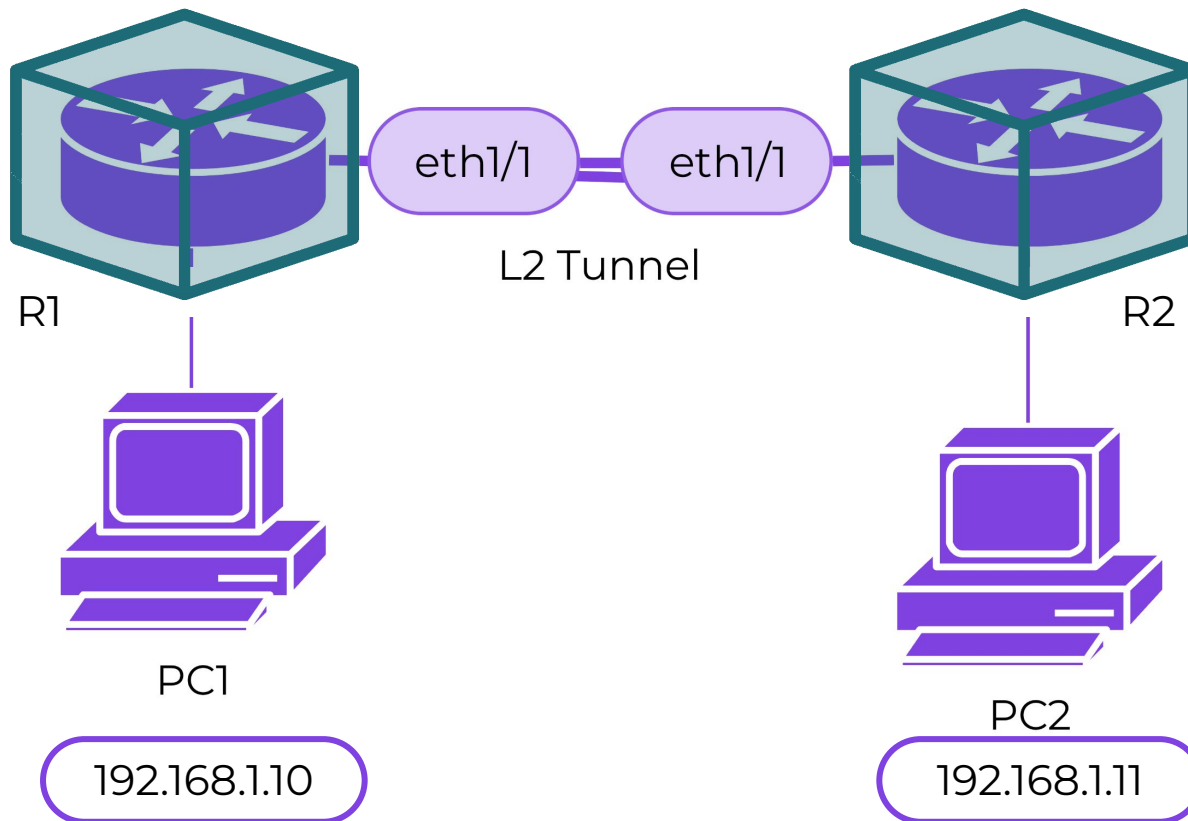

Код Containerlab

```
1.  rp := &clablinks.ResolveParams{
2.      Nodes: map[string]clablinks.Node{
3.          "host": clablinks.GetHostLinkNode(),
4.      },
5.      VxlanIfaceNameOverwrite: o.ToolsVxlan.Link,
6.  }
7.  link, err := vxlaw.Resolve(rp)
8.  if err != nil {
9.      return err
10. }
11. vxl, ok := link.(*clablinks.VxlanStitched)
12. if !ok {
13.     return fmt.Errorf("not a VxlanStitched link")
14. }
15. err = vxl.DeployWithExistingVeth(ctx)
16. if err != nil {
17.     return err
18. }
```

А если несколько worker, то VXLAN



А если несколько worker, то VXLAN



Оценка Containerlab

Плюсы

- 1 Виртуальная масштабируемая топология
- 2 Поддержка OCI-образов под эмуляцией QEMU
- 3 VXLAN, который поддерживает очень много подсетей
- 4 Модель прав на неймспейс в Kubernetes

Минусы

- 1 Нужно администрировать Kubernetes
- 2 GUI из коробки нет, но можно сделать свой
- 3 Избыточная схема для индивидуальных занятий

Мегасеть и мегавыводы

Выводы

- 1 Для физического оборудования — Terraform, Go и необходимые библиотеки.
- 2 PNETLab — для работы в закрытом контуре.
- 3 Containerlab — для построения крупных виртуализированных сетевых топологий и запуска OCI-образов контейнеров.

точка банк



Исходный код и презентация

Но NAT тоже костыльный

```
1.  apiVersion: networking.k8s.io/v1
2.  kind: NetworkPolicy
3.  metadata:
4.    name: deny-external-egress
5.    namespace: c9s-tests
6.  spec:
7.    podSelector: { }
8.    policyTypes:
9.      - Egress
10.   egress:
11.     - to:
12.       - ipBlock:
13.         cidr: 10.0.0.0/8
14.       - ipBlock:
15.         cidr: 10.43.0.0/16
16.       - ipBlock:
17.         cidr: 10.42.0.0/16
18.       - ipBlock:
19.         cidr: 172.0.0.0/8
20.     - to:
21.       - namespaceSelector:
22.         matchLabels:
23.           kubernetes.io/metadata.name: c9s
24.       - namespaceSelector:
25.         matchLabels:
26.           kubernetes.io/metadata.name: kube-system
```


Но NAT тоже костыльный

```
1.  apiVersion: networking.k8s.io/v1
2.  kind: NetworkPolicy
3.  metadata:
4.    name: deny-external-egress
5.    namespace: c9s-tests
6.  spec:
7.    podSelector: { }
8.    policyTypes:
9.      - Egress
10.    egress:
11.      - to:
12.        - ipBlock:
13.          cidr: 10.0.0.0/8
14.        - ipBlock:
15.          cidr: 10.43.0.0/16
16.        - ipBlock:
17.          cidr: 10.42.0.0/16
18.        - ipBlock:
19.          cidr: 172.0.0.0/8
20.      - to:
21.        - namespaceSelector:
22.          matchLabels:
23.            kubernetes.io/metadata.name: c9s
24.        - namespaceSelector:
25.          matchLabels:
26.            kubernetes.io/metadata.name: kube-system
```

Но NAT тоже костыльный

```
1.  apiVersion: networking.k8s.io/v1
2.  kind: NetworkPolicy
3.  metadata:
4.    name: deny-external-egress
5.    namespace: c9s-tests
6.  spec:
7.    podSelector: { }
8.    policyTypes:
9.      - Egress
10.  egress:
11.    - to:
12.      - ipBlock:
13.        cidr: 10.0.0.0/8
14.      - ipBlock:
15.        cidr: 10.43.0.0/16
16.      - ipBlock:
17.        cidr: 10.42.0.0/16
18.      - ipBlock:
19.        cidr: 172.0.0.0/8
20.    - to:
21.      - namespaceSelector:
22.        matchLabels:
23.          kubernetes.io/metadata.name: c9s
24.      - namespaceSelector:
25.        matchLabels:
26.          kubernetes.io/metadata.name: kube-system
```

```
1.  apiVersion: networking.k8s.io/v1
2.  kind: NetworkPolicy
3.  metadata:
4.    name: allow-external-ping
5.    namespace: c9s
6.  spec:
7.    podSelector:
8.      matchLabels:
9.        app.kubernetes.io/name: router
10.   policyTypes:
11.     - Egress
12.   egress:
13.     - to:
14.       - ipBlock:
15.         cidr: 8.8.8.8/32
```

Но NAT тоже костыльный

```
1.  apiVersion: networking.k8s.io/v1
2.  kind: NetworkPolicy
3.  metadata:
4.    name: deny-external-egress
5.    namespace: c9s-tests
6.  spec:
7.    podSelector: { }
8.    policyTypes:
9.      - Egress
10.  egress:
11.    - to:
12.      - ipBlock:
13.        cidr: 10.0.0.0/8
14.      - ipBlock:
15.        cidr: 10.43.0.0/16
16.      - ipBlock:
17.        cidr: 10.42.0.0/16
18.      - ipBlock:
19.        cidr: 172.0.0.0/8
20.    - to:
21.      - namespaceSelector:
22.        matchLabels:
23.          kubernetes.io/metadata.name: c9s
24.      - namespaceSelector:
25.        matchLabels:
26.          kubernetes.io/metadata.name: kube-system
```

```
1.  apiVersion: networking.k8s.io/v1
2.  kind: NetworkPolicy
3.  metadata:
4.    name: allow-external-ping
5.    namespace: c9s
6.  spec:
7.    podSelector:
8.      matchLabels:
9.        app.kubernetes.io/name: router
10.   policyTypes:
11.     - Egress
12.   egress:
13.     - to:
14.       - ipBlock:
15.         cidr: 8.8.8.8/32
```

Но NAT тоже костыльный

```
1.  apiVersion: networking.k8s.io/v1
2.  kind: NetworkPolicy
3.  metadata:
4.    name: deny-external-egress
5.    namespace: c9s-tests
6.  spec:
7.    podSelector: { }
8.    policyTypes:
9.      - Egress
10.  egress:
11.    - to:
12.      - ipBlock:
13.        cidr: 10.0.0.0/8
14.      - ipBlock:
15.        cidr: 10.43.0.0/16
16.      - ipBlock:
17.        cidr: 10.42.0.0/16
18.      - ipBlock:
19.        cidr: 172.0.0.0/8
20.    - to:
21.      - namespaceSelector:
22.        matchLabels:
23.          kubernetes.io/metadata.name: c9s
24.      - namespaceSelector:
25.        matchLabels:
26.          kubernetes.io/metadata.name: kube-system
```

```
1.  apiVersion: networking.k8s.io/v1
2.  kind: NetworkPolicy
3.  metadata:
4.    name: allow-external-ping
5.    namespace: c9s
6.  spec:
7.    podSelector:
8.      matchLabels:
9.        app.kubernetes.io/name: router
10.   policyTypes:
11.     - Egress
12.   egress:
13.     - to:
14.       - ipBlock:
15.         cidr: 8.8.8.8/32
```